HRMS

系統分析與架構總覽

HRMS System Analysis & Architecture

系統名稱
人力資源暨專案管理系統

版本
1.0

日期
2026-03-18

作者
HRMS 開發團隊

目錄

企業級人資暨專案管理系統 (HRMS) - 系統分析與架構設計說明書

本文件為 HRMS (Human Resources and Project Management System) 之核心設計文件，旨在提供開發團隊、維運團隊與系統利益關係人（Stakeholders）一份具體的系統總覽與架構規範。

一、 專案預期目標 (Project Objectives)

1. **整合化管理**：屏除企業過去各自獨立的考勤、薪資、招募與專案系統，建立統一的 Single-Source-of-Truth 人資入口。
2. **高可用性與擴展性**：採用微服務 (Microservices) 架構，確保在特定模組高併發或發生故障時，不影響全系統運行。
3. **符合法規與靈活的考勤計薪**：內建台灣勞基法計薪引擎，處理日新月異的變形工時與特殊休假日規則。
4. **提升招募與人才培育效率**：串接內部教育訓練、招募面試與績效考核流程，建立完整的人才生命週期管理。

二、 系統需求與核心功能模組 (System Requirements & Modules)

系統共分為 14 個獨立微服務，依照領域驅動設計 (DDD) 的 Bounded Context 劃分：

- **基礎核心領域 (Core Domain)**
  - 【01-IAM】：身分認證 (JWT/OAuth)、權限控制 (RBAC/ABAC)、組織登入隔離。
  - 【02-ORG】：組織樹建立、部門管理、職稱與員工基礎資料建立。
- **人力資源領域 (HR Domain)**
  - 【03-ATT】：考勤打卡、請假簽核、加班申請、特休排休、遲到早退判定。
  - 【04-PAY】：薪資結構與級距、每月薪資結算 (SAGA 交易)、獎金發放、媒體檔產出。
  - 【05-INS】：勞保、健保、勞退的加退保管理與級距自動對應與試算。
- **專案與績效領域 (PM & Performance Domain)**
  - 【06-PRJ】：專案生命週期、WBS 架構、工作包指派。
  - 【07-TMS】：專案人力成本投入追蹤、每週/週工時 (Timesheet) 填寫與多階簽核。
  - 【08-PFM】：OKR 與 KPI 設定、季度考核與 360 度評估評等。
- **招募與人才發展領域 (Talent Domain)**
  - 【09-RCT】：人才招募全流程、需求提報、面試排程規劃與錄取管理。

  - 【10-TRN】：企業內部教育訓練課程管理、證照管理與上課報名簽辦。
- **基礎設施與支援領域 (Support Domain)**
  - 【11-WFL】：視覺化表單與多層級動態簽核工作流引擎。
  - 【12-NTF】：系統事件通知，支援站內信、Email。
  - 【13-DOC】：企業內部文件知識庫、夾檔上傳。
  - 【14-RPT】：資料倉儲與主管戰情室 (Dashboard)，多維度資料交叉分析。

三、 雲端硬體與網路部署規劃 (Infrastructure & Networking)

本系統專為雲原生 (Cloud-Native) 環境設計，以 GCP (Google Cloud Platform) 作為基準部署架構範例。

1. 網路拓撲與資安分層 (Network Topology & Security)

- **【Tier 1】對外防護與負載均衡層 (Edge Layer)**：
  - WAF (Web Application Firewall) 防範 DDoS 與惡意 Request。
  - Cloud Load Balancer / API Gateway：統一 HTTPS 入口，處理 SSL 憑證卸載 (SSL Offloading)，執行 JWT 驗證，並基於 Path-based Routing 導向對應的微服務。
- **【Tier 2】應用程式層 (Application Layer)**：
  - 部署在 Cloud Run (Serverless) 或 GKE (Google Kubernetes Engine) 的私有子網 (Private Subnet)。
  - 各微服務之間透過 VPC 網域溝通，無對外的 Public IP。
- **【Tier 3】資料持久層 (Data Layer)**：
  - 完全處於內部網路 (Private IP only)，禁止外部直連。
  - 透過 Cloud SQL Auth Proxy 建立安全通道連線。

2. 硬體與資源配置建議 (Hardware Requirements)

(以下為以 GCP 為基準之 500 人企業規模估算)

- **後端運算實例 (Application Nodes)**：每個微服務建議配置 $0.5 \sim 1 \text{ vCPU}, 1\text{GB} \sim 2\text{GB RAM}$。高併發之考勤模組支援水平擴展。
- **主資料庫 (Cloud SQL for PostgreSQL)**：建議採用 High Availability (HA) 主從備援架構，$4 \text{ vCPU}, 16\text{GB RAM}$，初期配置 100GB SSD。
- **記憶體快取 (Redis / Memorystore)**：用於分散式 Session、權限樹狀快取、重試計數防護。建議配置 Standard Tier, $1\text{GB} \sim 2\text{GB}$ 容量。
- **訊息佇列 (Apache Kafka)**：建議由 Confluent Cloud 或自建多節點 Cluster，用以處理跨微服務的 Domain Event 吞吐量。

四、 系統整體架構圖 (System Architecture Diagram)

下圖展示系統自展示層 (Presentation) 經負載均衡，再進入微服務治理層，最終與基礎設施層的完整溝通拓撲，以及事件總線 (Event Bus) 之分佈。

> *圖表格式： PlantUML 渲染 | 原始碼： `00_系統分析與架構總覽.puml` | 渲染指令： `java -jar tools/plantuml.jar -smetana -o diagrams 00_系統分析與架構總覽.puml`*

HRMS 系統整體架構拓撲圖

五、 核心架構設計原則 (Architecture Philosophy)

1. **獨立資料庫與防腐層 (Database-per-service & Anti-corruption Layer)**： 每個微服務皆持有專屬之隔離性 PostgreSQL Schema，服務之間禁止跨庫 JOIN 表，杜絕系統死鎖與連鎖失效危機。各模組邊界由限界上下文 (Bounded Context) 嚴密定義。
2. **CQRS 控制流設計 (Command & Query Responsibility Segregation)**：
   - 更新性業務 (Command)：進入 `CommandApiService` 處理商業封裝規則並由資料層拋出事件。
   - 讀取性業務 (Query)：經由 `QueryApiService` 透過 QueryDSL/MyBatis 語系與 Read-Model 資料表互動，實現極致的讀取與分頁效能。
3. **最終一致性之非同步驅動 (Event-Driven Architecture & Eventual Consistency)**： 所有微服務建立之關鍵實體狀態變化 (如： `LeaveApprovedEvent`, `EmployeeTerminatedEvent`) 皆需拋送至 Kafka 主題通道。下游系統 (如：薪資結算) 透過訂閱通道自行運算更新，達成服務的完全非同步解耦設計。

六、 開發現況 (Development Status)

6.1 前端完成度

14 個微服務的前端頁面已**全部完整實作**（14/14 Complete），涵蓋所有核心業務功能與支援模組。

6.2 UI/UX 改版重點

系統已完成全面 UI/UX 改版，主要改善項目：

- **深色 Sidebar**：左側導覽列改為深色主題，提升視覺層次與專業感
- **登入頁重設計**：全新登入頁面視覺設計，強化品牌識別
- **統一 PageHeader**：9 個主要頁面套用統一的 PageHeader 元件，確保一致的操作體驗
- **頁面動畫與互動**：新增頁面切換動畫、通知鈴鐺、表格 hover 效果
- **Card 圓角與空狀態優化**：統一卡片圓角樣式，改善空資料狀態的顯示
- **404 頁面與登入導向**：新增 404 錯誤頁面，未登入時自動導向登入頁
- **確認對話框與搜尋欄統一**：危險操作前加入確認對話框，搜尋欄樣式全面統一

核心業務循序圖 (Core Sequence Diagrams)

本文件匯整了 HRMS 系統中最核心的系統循序圖 (Sequence Diagram)。透過這些循序圖，展示了本專案在**事件驅動架構 (Event-Driven Architecture)** 下的運作機制，說明如何解決傳統單體架構的強耦合問題，並達成微服務間的非同步協作與單點故障隔離。

一、 新進員工入職流程 (Employee Onboarding Flow)

此流程為展示微服務特性的經典場景。當人資 (HR) 建立一筆新員工資料時，系統**不會**讓 Organization (組織模組) 依序同步呼叫 IAM (權限)、Payroll (薪資)、Insurance (保險) 等服務，而是採取「發布 / 訂閱 (Pub/Sub)」模式進行非同步處理，大幅縮短了寫入資料的等待時間，並提高系統容錯率。

> *圖表格式： PlantUML 渲染 | 原始碼： `01_核心業務循序圖.puml` | 渲染指令： `java -jar tools/plantuml.jar -smetana -o diagrams 01_核心業務循序圖.puml`*

員工入職流程

流程設計要點 (Architecture Design Points)

此設計涵蓋了以下三個重要技術決策：

1. **避免微服務間同步呼叫（解決系統耦合）** 如果使用同步呼叫 (Synchronous REST Call)，當 IAM 或 Payroll 某一個服務無回應時，ORG 模組的新增操作就會跟著失敗並回滾。此外，未來若需增加新模組（例如資產管理模組發配電腦），ORG 的程式碼勢必面臨修改。 改用 Kafka 事件驅動後，ORG 僅負責『廣播』員工建檔完成，後續處理不影響 ORG 的主流程，完全符合**單一職責原則 (SRP)** 與**開閉原則 (OCP)**。
2. **解決高併發下的效能瓶頸（提升使用者體驗）** 建立帳號、配置保險、計算薪資結構乃至寄送 Email，整體流程可能耗時 3 至 5 秒。採用非同步處理後，ORG 寫入資料與將 Event 放進 Kafka 只需不到 50 毫秒 (ms)，使用端可瞬間收到成功回饋。其餘業務邏輯交由背景服務獨立且平行地處理，達到**最終一致性 (Eventual Consistency)**。
3. **分散式交易的事件不遺失保證（資料一致性）** 為了避免資料庫寫入成功，但 Kafka 服務異常導致事件漏發。在架構設計上採用 **Outbox Pattern (發件箱模式)**，將領域事件先寫入關聯式資料庫的 `outbox_events` 表，與業務操作同屬一個 Transaction，再由系統排程或 Debezium (CDC) 非同步將 Event 推送至 Kafka，以達成 **At-Least-Once (至少一次)** 投遞保證。

二、 請假簽核流程 (Leave Approval Flow)

此流程展示 **WFL（工作流引擎）** 與 **ATT（考勤模組）** 之間透過 Kafka 解耦的多階簽核設計。員工提交請假申請後，系統立即回應成功，並非同步觸發簽核任務指派與主管通知，最終依審核結果更新假期餘額。

請假簽核流程

流程設計要點

1. **ATT 與 WFL 完全解耦**：ATT 不直接呼叫 WFL，只需發布事件。未來若簽核規則改變（如增加 HR 複核），只需修改 WFL 的流程定義，ATT 零改動。
2. **餘額暫凍機制**：申請提交後立即凍結對應時數，防止同一時段被重複申請；駁回後自動解凍，不需人工介入。
3. **雙向事件確認**：核准和駁回各有獨立的 Domain Event，下游服務（ATT、NTF）各自訂閱並處理，流程可稽核、可重播。

三、 薪資月結 SAGA 補償流程 (Payroll Monthly Run with SAGA)

此流程展示 **SAGA Pattern** 如何在不使用分散式鎖（2PC）的前提下，確保薪資結算的資料一致性。薪資計算需跨 ATT、INS、PAY 三個服務聚合資料，任何步驟失敗都會觸發補償交易，不留中間污染狀態。

薪資SAGA補償流程

流程設計要點

1. **為何選 SAGA 而非 2PC**：跨微服務無法使用全域資料庫鎖。SAGA 的每個步驟只鎖自己的資料，失敗時透過補償交易逆向回滾，不阻塞其他服務。
2. **API Composition vs 事件驅動**：薪資結算需要「同步聚合」考勤與保費資料（當下最新數據），因此採用同步 REST API Composition，而非非同步事件訂閱。
3. **冪等性設計**：`payroll_runs` 以 `(year, month)` 為唯一鍵，重複觸發不會產生重複批次；補償交易使用 `DELETE WHERE run_id = ?` 確保冪等清除。

四、 員工離職流程 (Employee Offboarding Flow)

此流程為入職流程的鏡像，同樣展示事件驅動的**一事件多服務響應**模式。HR 只需執行一個「辦理離職」操作，後續的帳號停用、退保、最終薪資結算全部由各服務自主完成，無需人工逐一操作。

員工離職流程

流程設計要點

1. **與入職流程的對稱設計**：`EmployeeTerminatedEvent` 與 `EmployeeCreatedEvent` 結構對稱，各服務訂閱自己關心的事件並自主處理，ORG 不需要知道有哪些下游服務存在。
2. **帳號停用的時間差設計**：IAM 設定的是**排程停用**而非立即停用，允許員工在離職日當天完成最後的工作交接（如最後一次系統操作），避免即時停用造成業務中斷。
3. **未休特休折現計算**：PAY 在收到事件時先計算折現金額並標記，等到當月薪資結算時一併納入最終薪資，確保計算時機與薪資週期對齊，不造成額外的帳款。

五、 LDAP 登入與 JIT Provisioning 流程 (LDAP Login with JIT Provisioning)

此流程展示 **混合認證模式**：企業可同時支援 LDAP/AD 與本地帳密登入。LDAP 認證成功後，系統透過 **JIT (Just-In-Time) Provisioning** 自動在本地建立帳號，並同步 LDAP 群組到 RBAC 角色，無需手動在 HRMS 維護帳號。

LDAP登入流程

流程設計要點

1. **LDAP 優先、本地回退的混合認證**：`LoginServiceImpl` 先檢查是否啟用 LDAP。若啟用，優先透過 LDAP 認證；若 LDAP 伺服器不可達或使用者不存在於 AD，則自動回退至本地密碼比對，確保認證服務永不中斷。
2. **JIT Provisioning 自動建帳**：首次透過 LDAP 登入的使用者，系統自動在 IAM 建立對應的本地 `User` 記錄（authSource=LDAP、passwordHash=null），免除管理員預先建帳的人工成本。後續登入則同步最新的 displayName 與 email。
3. **LDAP 群組 → RBAC 角色自動映射**：`LdapGroupRoleMappingService` 依據組態 `groupRoleMapping` 將 AD 群組 DN 映射為 RBAC 角色代碼。每次 LDAP 登入都會重新同步，確保角色與 AD 組織異動即時一致。
4. **LDAP Injection 防護**：搜尋使用者 DN 時，透過 `escapeLdapFilter()` 轉義特殊字元（`*`, `(`, `)`, `\`, NUL），防止 LDAP 注入攻擊，符合資安規範。

六、 曠職自動判定流程 (Absent Detection Flow)

此流程展示 **排程驅動的異常偵測**模式。每日下班後由 `AbsentDetectionJob` 自動掃描全員出勤狀態，將「無打卡且無核准請假」的員工判定為曠職，並透過 Domain Event 通知主管與 HR，實現零人工介入的考勤異常管理。

曠職判定流程

流程設計要點

1. **三方交叉比對的領域邏輯**：`AbsentDetectionDomainService` 以「全員名冊 - 已打卡 - 已請假 = 缺勤」的集合運算判定曠職，邏輯純粹且可單元測試，不依賴框架。
2. **ReadModel 跨服務查詢**：在職員工清單來自 `employee_read_models` 表（CQRS ReadModel），避免 ATT 直接 REST 呼叫 ORG 微服務，降低服務間耦合。當 ReadModel 不可用時，降級為從 `leave_balances` 推導員工清單。
3. **事件驅動的後續處理**：每筆曠職記錄寫入後發布 `AttendanceAnomalyDetectedEvent`，下游 NTF 服務訂閱後即時推送通知給直屬主管與 HR，無需 ATT 關心通知邏輯。
4. **容錯設計**：逐筆員工建立缺勤記錄，單筆失敗（如重複記錄）不影響其餘員工的判定，並記錄 success/fail 計數供監控告警。

七、 薪資預借審核流程 (Salary Advance Approval Flow)

此流程展示 **多階審核 + 分期扣回** 的完整生命週期。員工提出薪資預借申請後，系統自動計算可借上限、由主管審核，核准後排程撥款，並在後續月薪結算時自動分期扣回，直到全額清償。

薪資預借審核流程

流程設計要點

1. **預借上限公式化驗證**：Domain 層內建 `calculateMaxAdvance()` 方法，以「(應發薪資 - 法定扣除 - 法扣款) x 90%」計算可借上限，確保員工不會過度預借導致薪資為負。
2. **聚合根狀態機**：`SalaryAdvance` 以 `AdvanceStatus` 列舉（PENDING → APPROVED → DISBURSED → REPAYING → FULLY_REPAID）管理生命週期，每次狀態轉移皆有前置狀態檢查，杜絕非法操作（如重複撥款、已撥款後取消）。
3. **WFL 簽核整合**：預借申請透過 Kafka 事件觸發 WFL 建立簽核任務，主管可在統一的簽核入口審核，與請假簽核共享相同的工作流引擎，降低維護成本。
4. **自動分期扣回**：PAY 每月薪資結算時自動查詢 DISBURSED/REPAYING 狀態的預借，呼叫 `SalaryAdvance.repay()` 扣回對應金額。最後一期自動調整（不超過剩餘餘額），扣完後狀態自動轉為 FULLY_REPAID。

核心實體關聯圖 (Global Entity Relationship Diagram)

本文件展示 HRMS 系統的跨服務邏輯實體關聯圖 (Logical ERD)。在微服務架構 (Microservices) 及「每服務一資料庫 (Database-per-service)」的設計下，不同模組之間**並不存在實體的外鍵約束 (No Physical Foreign Keys)**。跨模組的資料實體皆以 UUID 作為邏輯引用 (Logical Reference)，並透過事件驅動或 API Composition 進行資料聚合。

> *圖表格式： PlantUML 渲染 | 原始碼： `02_跨微服務實體關聯圖.puml` | 渲染指令： `java -jar tools/plantuml.jar -smetana -o diagrams 02_跨微服務實體關聯圖.puml`*

HRMS 跨微服務實體關聯圖

關聯設計要點 (Schema Design Principles)

本系統的資料架構具備以下三個高可用性與資安防護的設計特徵：

1. **跨微服務無實體外鍵（No Physical Foreign Keys）** 在傳統單體 (Monolith) 系統中，跨業務模組常倚賴實體關聯 (如：`Payroll.employee_id` 指向 `Employee` 表)。在本系統微服務架構中，薪資模組 (Payroll) 與人事模組 (Organization) 具備獨立的 PostgreSQL Schema，改以邏輯外鍵 (Logical Reference) 進行弱關聯 (如圖上虛線所示)。此舉能徹底隔離單一模組資料庫異常所引發的死鎖 (Deadlock) 與系統級存取失效。

2. **跨模組資料聚合策略 (Data Aggregation & API Composition)** 由於跨庫無法執行 SQL JOIN 語句，當前端要求複合性檢視表 (如：包含員工姓名與部門名稱的打卡列表) 時，系統採取 **CQRS 加上 API Composition** 的架構設計。在主領域模型撈取分頁資料後，利用 Application Service 發出批次查詢 (Batch Query) 獲取來自關聯模組的 Snapshot；針對高頻讀取的情境，則利用 Kafka 訂閱關聯實體的異動事件，於本地端維護讀取專用的快取模型 (Read-Model)。

3. **採用 UUID 構築無狀態主鍵設計** 所有微服務之領域實體皆強制採用 `UUID V4` 取代傳統的 Auto Increment ID。此策略除避免在跨微服務水平擴展時發生 ID 碰撞外，更天然地阻絕了 **ID 猜測攻擊 (BOLA: Broken Object Level Authorization)**，使得資料探勘與外洩風險降至最低。

---

14 服務核心實體總覽 (Core Entities per Service)

下表列出每個微服務的核心 Aggregate Root 與主要 Entity，以及其主鍵（PK）格式。

服務	Aggregate Root / Entity	PK 欄位	說明
HR01 IAM	`User`	`user_id` (UUID)	系統帳號，含密碼雜湊與角色
	`Role`	`role_id` (VARCHAR)	角色定義，含權限集合
	`Permission`	`permission_id` (VARCHAR)	系統權限（resource:action）
HR02 Organization	`Employee`	`employee_id` (UUID)	員工主檔（姓名、身分證、職稱、部門）
	`Department`	`department_id` (UUID)	部門結構（支援多層樹狀）
	`Organization`	`organization_id` (UUID)	公司/事業群頂層組織
HR03 Attendance	`AttendanceRecord`	`record_id` (UUID)	每日打卡紀錄（上下班時間）
	`LeaveRequest`	`request_id` (UUID)	請假申請（假別、起迄、狀態）
	`OvertimeRequest`	`overtime_id` (UUID)	加班申請（時數、核准狀態）
HR04 Payroll	`PayrollRun`	`run_id` (UUID)	計薪批次（月份、狀態、總額）
	`Payslip`	`payslip_id` (UUID)	個人薪資單（底薪、加項、扣項）
	`SalaryStructure`	`structure_id` (UUID)	薪資結構範本
HR05 Insurance	`Enrollment`	`enrollment_id` (UUID)	投保紀錄（勞保/健保/退休金）
	`InsuranceLevel`	`level_id` (UUID)	投保級距（金額區間、費率）
	`Contribution`	`contribution_id` (UUID)	保費分攤明細（雇主/員工/政府）
HR06 Project	`Project`	`project_id` (UUID)	專案主檔（客戶、狀態、預算）
	`Customer`	`customer_id` (UUID)	客戶主檔
	`WbsItem`	`wbs_id` (UUID)	WBS 工作分解（多層樹狀）

服務	Aggregate Root / Entity	PK 欄位	說明
HR07 Timesheet	`WeeklyTimesheet`	`timesheet_id` (UUID)	週報主表（員工、週次、狀態）
	`TimesheetEntry`	`entry_id` (UUID)	工時明細（專案、日期、時數）
HR08 Performance	`ReviewCycle`	`cycle_id` (UUID)	考核週期（期間、權重、狀態）
	`ReviewForm`	`form_id` (UUID)	個人考核表（自評、主管評、評等）
	`ReviewTemplate`	`template_id` (UUID)	考核範本（項目、配分）
HR09 Recruitment	`JobOpening`	`opening_id` (UUID)	職缺（名稱、需求人數、狀態）
	`Application`	`application_id` (UUID)	應徵紀錄（候選人、進度）
	`Interview`	`interview_id` (UUID)	面試排程（時間、面試官、結果）
HR10 Training	`Course`	`course_id` (UUID)	課程（名稱、講師、時數）
	`Enrollment`	`enrollment_id` (UUID)	報名紀錄（員工、課程、狀態）
	`Certificate`	`certificate_id` (UUID)	證照紀錄（名稱、到期日）
HR11 Workflow	`ProcessDefinition`	`definition_id` (UUID)	流程定義（節點、路由、條件）
	`ProcessInstance`	`instance_id` (UUID)	流程實例（狀態、當前節點）
	`ApprovalTask`	`task_id` (UUID)	簽核任務（簽核人、結果）
HR12 Notification	`Notification`	`notification_id` (UUID)	通知紀錄（類型、標題、狀態）
	`NotificationTemplate`	`template_id` (UUID)	通知範本（頻道、內容模板）
HR13 Document	`Document`	`document_id` (UUID)	文件主檔（名稱、路徑、版本）

服務	Aggregate Root / Entity	PK 欄位	說明
	`DocumentVersion`	`version_id` (UUID)	版本歷程（版號、上傳者）
	`DocumentTemplate`	`template_id` (UUID)	文件範本（範本類型、欄位定義）
HR14 Reporting	`ReportDefinition`	`report_id` (UUID)	報表定義（類型、資料來源）
	`DashboardWidget`	`widget_id` (UUID)	儀表板元件（圖表類型、查詢）
	`ScheduledReport`	`schedule_id` (UUID)	排程報表（週期、收件人）

跨服務引用關係圖 (Cross-Service Reference Map)

下圖展示各服務之間的 UUID 邏輯引用關係。所有引用均為 **邏輯外鍵（Logical Reference）**，無物理 FK 約束。

PlantUML 渲染失敗

核心共用 UUID 引用彙整

UUID 類型	來源服務	引用服務	用途
`employee_id`	HR02 Organization	HR01, HR03, HR04, HR05, HR07, HR08, HR09, HR10	**最核心的跨服務 ID**，幾乎所有服務都引用
`department_id`	HR02 Organization	HR03, HR08, HR14	部門篩選、團隊績效、部門報表
`user_id`	HR01 IAM	HR11, HR12, HR13	簽核人、通知收件人、文件上傳者
`project_id`	HR06 Project	HR07 Timesheet	工時歸屬專案
`customer_id`	HR06 Project	HR14 Reporting	客戶專案報表
`wbs_id`	HR06 Project	HR07 Timesheet	工時歸屬 WBS 項目

API Composition 實例

實例一：報表儀表板 (Dashboard Composition)

HR14 Reporting 服務的儀表板需從多個服務聚合資料：

```
┌──────────────────────────────────────────────────────┐
│              HR14 Reporting Service                  │
│                 Dashboard API                        │
├──────────────────────────────────────────────────────┤
│                                                      │
│  GET /api/v1/reports/dashboard                       │
│                                                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐          │
│  │ 員工統計  │  │ 考勤摘要  │  │ 薪資總覽  │          │
│  │ ReadModel │  │ ReadModel │  │ ReadModel │          │
│  └────┬─────┘  └────┬─────┘  └────┬─────┘          │
│       │             │             │                  │
└───────┼─────────────┼─────────────┼──────────────────┘
        │             │             │
   ┌────▼────┐   ┌────▼────┐   ┌────▼────┐
   │ HR02 ORG │   │ HR03 ATT │   │ HR04 PAY │
   │ Query API│   │ Query API│   │ Query API│
   └─────────┘   └─────────┘   └─────────┘
```

組合方式：

1. CQRS ReadModel：訂閱各服務 Domain Event，在本地維護 Materialized View
2. API Composition：即時查詢各服務 Query API，在 Application Service 層合併
3. 混合模式：高頻資料用 ReadModel，低頻資料用即時查詢

實例二：員工 360 度檢視 (Employee 360 View)

員工個人頁面需聚合來自 8 個服務的資料：

區塊	來源服務	API	說明
基本資料	HR02 Organization	`GET /api/v1/employees/{id}`	姓名、部門、職稱
帳號狀態	HR01 IAM	`GET /api/v1/users/{id}`	登入狀態、角色
本月考勤	HR03 Attendance	`GET /api/v1/attendance/my-records`	出勤率、遲到次數
薪資單	HR04 Payroll	`GET /api/v1/payslips/latest`	最近薪資
保險資料	HR05 Insurance	`GET /api/v1/insurance/my`	投保級距
本週工時	HR07 Timesheet	`GET /api/v1/timesheets/my`	本週已填工時
績效狀態	HR08 Performance	`GET /api/v1/performance/my`	當前週期評核進度
訓練紀錄	HR10 Training	`GET /api/v1/training/my`	已完成課程數

系統使用案例圖與規格說明 (Use Case Model & Descriptions)

本文件定義了 HRMS (人資暨專案管理系統) 的核心參與者 (Actors) 及其可執行的系統操作 (Use Cases)，以確保系統功能邊界符合各層級使用者的業務需求。

一、 系統參與者定義 (Actors)

系統基於 RBAC (Role-Based Access Control) 設計，主要分為三大參與角色：

1. **一般員工 (Employee / ESS User)**：系統的基層使用者，可存取 ESS (Employee Self-Service) 功能。
2. **部門主管 (Manager)**：擁有下屬管理權限的員工，負責審核與進度控管。
3. **人資/系統管理員 (HR / Admin)**：擁有全域管理權限，負責組織架構維護、系統參數設定與薪資結算。

二、 核心系統使用案例圖 (Core Use Case Diagram)

下圖展示了不同參與者在系統中的核心操作與權限交集。

> ***圖表格式說明：*** *本文件所有圖表均以 PlantUML 渲染（純 Java 引擎，免裝 GraphViz）。*
>
> - *Use Case Diagram 原始碼：* `03_系統使用案例圖與規格.puml`
> - *Activity Diagrams (UC01~UC12) 原始碼：* `03_系統使用案例圖與規格_活動圖.puml`
> - *渲染指令：* `java -jar tools/plantuml.jar -smetana -o diagrams <puml檔案>`

HRMS 核心使用案例圖

三、 核心案例規格描述 (Use Case Descriptions)

針對關鍵之使用案例，提供詳細的事件流 (Flow of Events) 與前置/後置條件。

[UC01] 個人資料與帳號維護 (Maintain Personal Profile & Account)

- **參與者 (Actor)**：一般員工
- **前置條件 (Preconditions)**：員工已登入系統，具備有效在職狀態。
- **主要流程 (Main Flow)**：
  1. 員工進入「個人設定」模組，系統顯示目前基本資料（聯絡資訊、緊急聯絡人、學歷、證照等）。

2. 員工修改所需欄位後點擊提交。
3. 系統驗證格式（手機格式、Email 格式等）。
4. 系統儲存變更並寫入稽核日誌。
5. 若員工欲修改登入密碼，需輸入舊密碼驗證後方可設定新密碼（密碼須符合複雜度規則）。

- **替代流程 (Alternative Flow)**：
  - 若欲修改需 HR 審核的核心欄位（如姓名、身份證字號），系統自動轉發審核流程，待 HR 核准後生效。
- **後置條件 (Postconditions)**：個人資料更新完成，相關下游服務（如【NTF 通知模組】）同步接收最新聯絡資訊。

活動圖：

UC01 活動圖

[UC02] 每日考勤打卡 (Daily Clock-In / Clock-Out)

- **參與者 (Actor)**：一般員工
- **前置條件 (Preconditions)**：員工已登入系統；系統時間在可打卡時間範圍內（依班表設定）。
- **主要流程 (Main Flow)**：
  1. 員工進入「考勤打卡」頁面，系統顯示今日班表與目前打卡狀態。
  2. 員工點擊「上班打卡」或「下班打卡」。
  3. 系統記錄打卡時間與 IP 位址（若啟用地理限制則驗證位置）。
  4. 系統與排定班表比對，自動判定是否遲到或早退，並即時顯示結果（正常 / 遲到 / 早退）。
- **替代流程 (Alternative Flow)**：
  - 若員工忘記打卡，可申請「補打卡」，填具原因後由主管審核生效。
- **異常流程 (Exception Flow)**：
  - 若系統偵測到 IP 不在允許範圍（如遠端辦公），仍允許打卡但標記為「異常待確認」，並自動通知主管。
- **後置條件 (Postconditions)**：打卡紀錄儲存至考勤資料庫；若遲到超過閾值，系統自動觸發通知給直屬主管。

活動圖：

UC02 活動圖

[UC03] 發起請假/加班申請 (Submit Leave/Overtime Request)

- **參與者 (Actor)**：一般員工
- **前置條件 (Preconditions)**：員工已登入系統，且具備有效之在職狀態；當年度仍有可用之請假餘額(若是特休)。
- **主要流程 (Main Flow)**：

1. 員工進入「差勤申請」模組。
2. 系統帶出此員工目前各假別之可用餘額。
3. 員工選擇假別 (或加班類別)、輸入起始與結束時間，並上傳證明附件 (選填)。
4. 員工提交申請。
5. 系統驗證：確認時間未與其他表單重疊、餘額充足。
6. 系統建立表單紀錄，並透過【Workflow 引擎】匹配該員工所在部門之直屬主管為簽核者。
7. 系統發送通知 (經由【NTF 模組】) 給對應之主管。

- **後置條件 (Postconditions)**：請假單狀態變更為 `PENDING_APPROVAL` ；對應時數額度暫時凍結 (Locked)。

活動圖：

UC03 活動圖

[UC04] 查詢個人薪資單 (View Personal Payslip)

- **參與者 (Actor)**：一般員工（HR 管理員可查詢全員）
- **前置條件 (Preconditions)**：員工已登入系統；欲查詢月份的薪資結算批次已完成（狀態為 `COMPLETED` ）。
- **主要流程 (Main Flow)**：
  1. 員工進入「薪資單查詢」模組，系統列出所有歷史薪資月份清單。
  2. 員工選擇欲查閱之月份。
  3. 系統顯示詳細薪資明細：底薪、各項加給、加班費、請假扣薪、勞健保自付額、所得稅預扣、實領淨薪。
  4. 員工可下載 PDF 版電子薪資單留存。
- **替代流程 (Alternative Flow)**：
  - HR 管理員可透過「薪資管理」介面搜尋特定員工或部門的歷史薪資，進行全員薪資查詢。
- **異常流程 (Exception Flow)**：
  - 若當月薪資尚未結算，系統顯示「結算中，預計發放日為 XX/XX」，不顯示試算數字。
- **後置條件 (Postconditions)**：無資料變更；查詢操作寫入存取稽核日誌（DOC / RPT 模組）。

活動圖：

UC04 活動圖

[UC05] 審核下屬差勤表單 (Approve Subordinate Attendance Forms)

- **參與者 (Actor)**：部門主管
- **前置條件 (Preconditions)**：主管已登入系統；其下屬已提交請假或加班申請，狀態為 `PENDING_APPROVAL` 。

- **主要流程 (Main Flow)**：
  1. 主管進入「待審核」任務列表，系統列出所有等待其簽核的差勤表單。
  2. 主管選擇一筆表單，查看申請詳情（假別、時段、剩餘餘額、備註、附件）。
  3. 主管選擇「核准」或「駁回」，並填寫審核意見（駁回時為必填）。
  4. 系統更新表單狀態，並透過【NTF 模組】發送審核結果通知給申請員工。
  5. 若核准：正式扣除請假額度，考勤統計即時更新。
  6. 若駁回：還原暫凍額度，申請員工可修改後重新提交。
- **替代流程 (Alternative Flow)**：
  - 若主管長期不在，HR 可設定代理人審核，代理人具備相同審核權限。
- **後置條件 (Postconditions)**：請假紀錄依核准結果正式生效；月底薪資結算時自動計入請假扣薪。

活動圖：

UC05 活動圖

[UC06] 指派專案工作包 (Assign Project Work Package)

- **參與者 (Actor)**：部門主管 / 專案經理
- **前置條件 (Preconditions)**：專案已建立並存在有效的 WBS (Work Breakdown Structure)；受指派員工處於在職狀態。
- **主要流程 (Main Flow)**：
  1. 主管進入「專案管理」模組，選擇目標專案，系統展示 WBS 樹狀結構。
  2. 主管選擇欲指派的工作包 (Work Package)，系統顯示目前指派狀態與預算使用率。
  3. 主管從部門人員清單選擇負責人員，設定預計工時與截止日期。
  4. 系統建立指派記錄，更新工作包狀態為 `ASSIGNED`，並透過【NTF 模組】通知受指派員工。
- **替代流程 (Alternative Flow)**：
  - 若工作包已有指派對象，重新指派後，舊指派記錄自動標記為 `REASSIGNED`，歷史工時紀錄保留。
- **後置條件 (Postconditions)**：受指派員工可在「我的工作」頁面看到新任務，並可開始填寫 Timesheet 進行工時回報。

活動圖：

UC06 活動圖

[UC07] 進行下屬績效考核 (Conduct Performance Review)

- **參與者 (Actor)**：部門主管
- **前置條件 (Preconditions)**：績效考核週期已由 HR 啟動（如：2026 Q1 考核）；受評員工處於在職狀態。

- **主要流程 (Main Flow)**：
  1. 主管進入「績效考核」模組，系統列出本週期需評核的下屬清單。
  2. 主管選擇一位員工，查看該員工的 OKR/KPI 目標與自評結果。
  3. 主管填寫各評核項目之評分（工作品質、協作能力、目標達成率等）。
  4. 若啟用 360 度評估，系統彙整同儕互評結果供主管參考。
  5. 主管填寫整體評語並選擇最終評等（A / B / C / D 等）後提交。
  6. 系統通知員工查閱考核結果（依考核政策決定是否公開細項）。
- **替代流程 (Alternative Flow)**：
  - 員工對考核結果不服，可在申覆期限內提出申覆，系統啟動二次複評流程，由 HR 介入仲裁。
- **後置條件 (Postconditions)**：考核結果儲存，並可連動後續薪資調整與晉升作業。

活動圖：

UC07 活動圖

[UC08] 員工到職/離職建檔 (Employee Onboarding / Offboarding)

- **參與者 (Actor)**：人資管理員
- **前置條件 (Preconditions)**：到職：招募流程已完成，錄取通知已發出。離職：員工已提交離職申請或勞資協議終止。
- **主要流程 (到職 Onboarding)**：
  1. HR 進入「員工建檔」模組，填寫新員工基本資料（姓名、身份證、聯絡方式、到職日、部門、職稱、薪資結構）。
  2. HR 提交後，系統產生員工編號，並透過 Kafka 發布 `EmployeeCreatedEvent`。
  3. 下游服務自動響應：【IAM】建立登入帳號→發送報到通知；【INS】辦理勞健保加保；【PAY】建立薪資結構。
- **主要流程 (離職 Offboarding)**：
  1. HR 進入員工資料頁，執行「辦理離職」，填寫離職日期、離職原因、工作交接人。
  2. 系統透過 Kafka 發布 `EmployeeTerminatedEvent`。
  3. 下游服務自動響應：【IAM】於離職日停用帳號；【INS】辦理退保；【PAY】結算最終薪資（含未休特休折現）。
- **後置條件 (Postconditions)**：員工狀態更新為 `ACTIVE` 或 `TERMINATED`，所有下游服務自動同步，無需 HR 逐一操作。

活動圖：

UC08 活動圖

[UC09] 設定班表與假別參數 (Configure Shift Schedule & Leave Types)

- **參與者 (Actor)**：人資管理員

- **前置條件 (Preconditions)**：HR 已登入系統，具備系統參數維護權限。
- **主要流程 (班表設定)**：
  1. HR 進入「班表管理」模組，建立或修改班別（正常班、早班、晚班、彈性班等）。
  2. 設定各班別的上下班時間、彈性打卡區間、強制休息時間。
  3. 指定部門或個別員工適用之班別，設定生效日期。
- **主要流程 (假別設定)**：
  1. HR 進入「假別參數」模組，設定假別清單（特休、病假、事假、公傷假等）。
  2. 設定各假別的年度給假天數、可否跨年度累計、折算規則（如：未休特休時薪折算）。
  3. 設定國定假日行事曆，包含勞基法規定之補班補假對應規則。
- **後置條件 (Postconditions)**：新設定立即生效，影響後續所有員工的考勤判定邏輯與請假餘額計算。

活動圖：

UC09 活動圖

[UC10] 維護勞健保級距表 (Maintain Labor/Health Insurance Grade Table)

- **參與者 (Actor)**：人資管理員
- **前置條件 (Preconditions)**：HR 已登入系統，具備系統參數維護權限；政府已公告最新費率表（通常每年 1 月或 7 月調整）。
- **主要流程 (Main Flow)**：
  1. HR 進入「勞健保設定」模組，選擇欲更新的費率年度。
  2. 系統顯示目前各級距的投保薪資範圍、勞保費率、健保費率、勞退提撥比率。
  3. HR 依照政府公告，逐一更新各項費率或調整投保薪資級距表。
  4. HR 執行「批次試算」，預覽費率調整對全體員工每月保費之影響（顯示差異金額）。
  5. HR 確認無誤後儲存，設定新生效日期。
- **異常流程 (Exception Flow)**：
  - 若新費率或新級距低於員工目前投保薪資，系統警示需人工確認是否降級調整，避免違反勞基法最低投保規定。
- **後置條件 (Postconditions)**：新費率於設定之生效日起套用至薪資結算；歷史費率紀錄保留，不影響已發放薪資。

活動圖：

UC10 活動圖

[UC11] 執行每月薪資結算 (Execute Monthly Payroll Run)

- **參與者 (Actor)**：人資主管 / 系統管理員
- **前置條件 (Preconditions)**：考勤結算週期已結束，且當月所有請假、加班表單皆已簽核完畢。

- **主要流程 (Main Flow)**：
  1. HR 進入「薪資結算」模組，選擇結算計薪年月 (如：2026-02)。
  2. 系統建立一筆結算批次 (Payroll Run)，狀態為 `PROCESSING`。
  3. 系統透過 CQRS 與 API Composition，自動向【ATT 模組】拉取所有員工該月之請假扣薪與加班費加總。
  4. 系統依據【INS 模組】之勞健保與勞退設定，試算各員工之自付額。
  5. 薪資計算引擎 (SAGA Pattern) 依序算出每位員工之 Base Salary, Allowances, Deductions 與 Net Pay。
  6. 計算完畢後提供「發放前試算表」供 HR 預覽與微調。
  7. HR 確認無誤後點擊「鎖定與發放」。
  8. 系統產出各大銀行之薪資轉帳媒體檔 (TXT/CSV)，並產生個別員工之電子薪水單 (PDF)。
- **後置條件 (Postconditions)**：批次狀態變更為 `COMPLETED`；員工可於 ESS 自助服務系統中查閱該月電子薪資單。

活動圖：

UC11 活動圖

[UC12] 權限角色與工作流配置 (Configure RBAC Roles & Workflow)

- **參與者 (Actor)**：系統管理員 (Admin)
- **前置條件 (Preconditions)**：Admin 已登入系統並具備最高管理權限。
- **主要流程 (角色權限設定)**：
  1. Admin 進入「角色管理」模組，查看現有角色清單（一般員工、部門主管、HR、財務、系統管理員等）。
  2. Admin 建立新角色或修改現有角色，針對每個功能模組設定可操作的動作（查詢 / 建立 / 修改 / 刪除）。
  3. Admin 將角色指派給特定員工帳號或部門群組。
  4. 系統儲存設定，權限變更立即生效。
- **主要流程 (工作流設定)**：
  1. Admin 進入「流程設計器」，使用可視化工具建立或修改審核流程。
  2. 設定流程節點（如：請假簽核→員工提交→直屬主管核准→HR 備查）。
  3. 設定各節點的條件規則（如：請假超過 3 天需雙重核准；加班費超過 NT$5,000 需財務複核）。
  4. 發布流程設定，新提交的表單立即套用新流程；進行中的表單沿用舊流程。
- **替代流程 (Alternative Flow)**：
  - Admin 可匯出/匯入角色與流程設定，供多租戶環境複製使用。
- **後置條件 (Postconditions)**：新角色權限與簽核流程立即套用，影響所有相關使用者的操作範圍與表單路由。

活動圖：

UC12 活動圖

四、 系統管理使用案例 (System Administration Use Cases)

以下使用案例描述系統管理員對 HRMS 平台層級的管理操作，涵蓋功能開關、系統參數與排程任務三大面向。

[UC-SYS-01] 功能開關管理 (Manage Feature Toggles)

- **參與者 (Actor)**：系統管理員 (Admin)
- **前置條件 (Preconditions)**：Admin 已登入系統並具備 `SYSTEM_ADMIN` 權限；功能開關表（`feature_toggles`）已初始化。
- **主要流程 (Main Flow)**：
  1. Admin 進入「系統管理 > 功能開關」模組，系統列出所有功能開關項目，包含功能代碼（`featureCode`）、功能名稱、所屬模組（`HR01` ~ `HR14`）、目前啟停狀態。
  2. Admin 可依模組篩選或依關鍵字搜尋特定功能。
  3. Admin 選擇一筆功能開關，點擊「啟用」或「停用」按鈕進行切換。
  4. 系統呼叫 `FeatureToggle.toggle(operator)` 更新狀態與異動時間/異動人。
  5. 系統儲存變更並寫入稽核日誌。
  6. 功能開關狀態即時生效（不需重啟服務），影響對應模組的業務流程判斷。
- **替代流程 (Alternative Flow)**：
  - Admin 可批次啟用或停用同一模組下的所有功能開關，系統逐筆更新並記錄。
  - Admin 新增功能開關時，需填寫功能代碼（唯一）、功能名稱、所屬模組與說明，租戶 ID 由系統自動帶入。
- **例外流程 (Exception Flow)**：
  - 若功能代碼已存在，系統回傳 `409 Conflict` 錯誤，拒絕重複建立。
  - 若操作者不具備 `SYSTEM_ADMIN` 權限，系統回傳 `403 Forbidden`。
- **商業規則 (Business Rules)**：
  - BR-SYS-01-01：功能開關以 `featureCode` 為唯一識別，同一租戶下不可重複。
  - BR-SYS-01-02：多租戶環境下，每個租戶的功能開關獨立管理（依 `tenantId` 隔離）。
  - BR-SYS-01-03：功能開關異動必須記錄操作者與時間戳，供稽核查閱。
- **後置條件 (Postconditions)**：功能開關狀態更新完成，各模組即時讀取最新狀態進行業務邏輯判斷（如考勤遲到檢查 `ATTENDANCE_LATE_CHECK`、薪資預支 `SALARY_ADVANCE` 等）。

活動圖：

UC-SYS-01 活動圖

[UC-SYS-02] 系統參數管理 (Manage System Parameters)

- **參與者 (Actor)**：系統管理員 (Admin)
- **前置條件 (Preconditions)**：Admin 已登入系統並具備 `SYSTEM_ADMIN` 權限；系統參數表（`system_parameters`）已初始化。
- **主要流程 (Main Flow)**：
  1. Admin 進入「系統管理 > 系統參數」模組，系統列出所有參數項目，包含參數代碼（`paramCode`）、參數名稱、目前值、參數型別（`STRING` / `INTEGER` / `DECIMAL` / `BOOLEAN` / `JSON`）、所屬模組、參數分類（`SECURITY` / `BUSINESS` / `UI` / `SYSTEM`）。
  2. Admin 可依模組或分類篩選參數，或依關鍵字搜尋。
  3. Admin 選擇欲修改的參數，輸入新的參數值。
  4. 系統呼叫 `SystemParameter.updateValue(newValue, operator)`，回傳 `ParameterChange` 記錄（含舊值、新值、操作者、時間）。
  5. 系統驗證新值是否符合參數型別格式（如 `INTEGER` 型別需為合法整數）。
  6. 系統儲存變更，異動記錄寫入稽核日誌。
- **替代流程 (Alternative Flow)**：
  - Admin 可點擊「重設為預設值」，系統呼叫 `SystemParameter.resetToDefault(operator)` 將參數還原至出廠設定。
  - Admin 新增系統參數時，需填寫參數代碼（唯一）、名稱、預設值、型別、所屬模組、分類、說明，以及是否加密儲存。
  - 若參數標記為加密（`isEncrypted = true`），系統以密文形式儲存，查詢時遮罩顯示。
- **例外流程 (Exception Flow)**：
  - 若新值格式不符合參數型別定義（如 `BOOLEAN` 輸入非 `true/false`），系統回傳 `400 Bad Request`。
  - 若參數代碼已存在，系統回傳 `409 Conflict`。
  - 若操作者不具備 `SYSTEM_ADMIN` 權限，系統回傳 `403 Forbidden`。
- **商業規則 (Business Rules)**：
  - BR-SYS-02-01：參數以 `paramCode` 為唯一識別，同一租戶下不可重複。
  - BR-SYS-02-02：安全類參數（`category = SECURITY`，如 `MAX_FAILED_LOGIN_ATTEMPTS`）異動時，系統需發送通知給所有系統管理員。
  - BR-SYS-02-03：加密參數的舊值在異動記錄中也以遮罩形式記錄，避免敏感資訊外洩。
  - BR-SYS-02-04：多租戶環境下，`GLOBAL` 模組參數為共享參數，各租戶僅可覆寫自身範圍的參數。
- **後置條件 (Postconditions)**：系統參數更新完成並立即生效，影響對應模組的行為（如登入失敗上限、密碼複雜度規則、薪資計算參數等）。

活動圖：

UC-SYS-02 活動圖

[UC-SYS-03] 排程任務管理 (Manage Scheduled Job Configuration)

- **參與者 (Actor)**：系統管理員 (Admin)
- **前置條件 (Preconditions)**：Admin 已登入系統並具備 `SYSTEM_ADMIN` 權限；排程任務配置表已初始化。
- **主要流程 (Main Flow)**：
  1. Admin 進入「系統管理 > 排程任務」模組，系統列出所有排程任務項目，包含任務代碼（`jobCode`）、任務名稱、所屬模組、Cron 表達式、啟停狀態、最近執行時間、執行結果（`SUCCESS` / `FAILED` / `RUNNING`）、連續失敗次數。
  2. Admin 可依模組篩選或依執行狀態（正常/告警/停用）篩選。
  3. Admin 選擇一筆排程任務，可進行以下操作：
     - **啟用/停用**：呼叫 `ScheduledJobConfig.enable(operator)` 或 `disable(operator)`。
     - **修改 Cron 表達式**：呼叫 `ScheduledJobConfig.updateCron(newCron, operator)` 調整執行頻率。
     - **手動觸發**：立即執行一次該排程任務（不影響原排程週期）。
  4. 系統驗證 Cron 表達式格式正確性。
  5. 系統儲存變更並寫入稽核日誌。
- **替代流程 (Alternative Flow)**：
  - Admin 可查看特定任務的歷史執行記錄（含執行時間、耗時、結果、錯誤訊息），進行問題排查。
  - 若連續失敗次數 >= 3（`needsAlert() = true`），系統自動在任務列表中標示告警圖示，提醒 Admin 介入處理。
- **例外流程 (Exception Flow)**：
  - 若 Cron 表達式格式無效，系統回傳 `400 Bad Request`，不儲存變更。
  - 若手動觸發時任務已在執行中（`status = RUNNING`），系統回傳 `409 Conflict`，防止重複執行。
  - 若操作者不具備 `SYSTEM_ADMIN` 權限，系統回傳 `403 Forbidden`。
- **商業規則 (Business Rules)**：
  - BR-SYS-03-01：排程任務以 `jobCode` 為唯一識別。
  - BR-SYS-03-02：連續失敗 >= 3 次時，系統自動發送告警通知（透過 NTF 模組）給所有系統管理員。
  - BR-SYS-03-03：排程任務的執行歷史保留 90 天，超過自動清除。
  - BR-SYS-03-04：常見排程任務包含：考勤日結（`ATT_DAILY_CLOSE`）、薪資月結（`PAY_MONTHLY_RUN`）、保險申報（`INS_MONTHLY_REPORT`）、特休年結（`ATT_ANNUAL_LEAVE_RESET`）。
- **後置條件 (Postconditions)**：排程任務配置更新完成，排程引擎依新設定執行後續任務。

活動圖：

UC-SYS-03 活動圖

五、 輪班排程使用案例 (Shift Scheduling Use Cases)

以下使用案例描述班別定義、排班管理與換班申請的完整流程，對應 HR03 考勤模組中的輪班排程子系統。

[UC-ATT-SHIFT-01] 班別定義 (Define Shift Types)

- **參與者 (Actor)**：人資管理員 (HR/Admin)
- **前置條件 (Preconditions)**：HR 已登入系統並具備考勤管理權限；組織架構已建立。
- **主要流程 (Main Flow)**：
  1. HR 進入「考勤管理 > 班別管理」模組，系統列出目前所有已定義的班別，包含班別代碼（`code`）、名稱、班別類型（`REGULAR` / `FLEXIBLE` / `SHIFT`）、上下班時間、啟停狀態。
  2. HR 點擊「新增班別」，填寫以下資訊：
     - 班別代碼（唯一）、班別名稱
     - 班別類型：常規班（`REGULAR`）、彈性班（`FLEXIBLE`）、排班制（`SHIFT`）
     - 上班時間（`workStartTime`）、下班時間（`workEndTime`）
     - 休息時間起迄（`breakStartTime` 、 `breakEndTime`）（選填）
     - 遲到容忍分鐘數（`lateToleranceMinutes`）、早退容忍分鐘數（`earlyLeaveToleranceMinutes`）
     - 是否啟用遲到判定（`lateCheckEnabled`）、是否遲到扣薪（`lateSalaryDeduction`）
  3. HR 提交後，系統建立 `Shift` 聚合根，執行欄位驗證。
  4. 系統儲存班別資料，初始狀態為啟用（`isActive = true`）。
- **替代流程 (Alternative Flow)**：
  - HR 可修改現有班別的各項設定（呼叫 `PUT /api/v1/shifts/{shiftId}` ）。
  - HR 可停用班別（呼叫 `PUT /api/v1/shifts/{shiftId}/deactivate` ），已停用的班別不可被排班引用。
  - HR 可重新啟用已停用的班別。
- **例外流程 (Exception Flow)**：
  - 若班別代碼已存在，系統回傳 `409 Conflict` 。
  - 若上班時間或下班時間為空，系統回傳 `400 Bad Request` （ `Shift name/code/type/time cannot be empty/null` ）。
  - 若休息開始時間晚於結束時間，系統回傳 `400 Bad Request` （ `Break start time cannot be after break end time` ）。
  - 若容忍分鐘數為負數，系統回傳 `400 Bad Request` （ `Tolerance minutes cannot be negative` ）。
- **商業規則 (Business Rules)**：
  - BR-ATT-SHIFT-01-01：班別代碼在同一組織下唯一。
  - BR-ATT-SHIFT-01-02：已被排班引用的班別不可刪除，只能停用。
  - BR-ATT-SHIFT-01-03：彈性班（`FLEXIBLE`）的遲到容忍分鐘數建議設為 30~60 分鐘。

  - BR-ATT-SHIFT-01-04：班別的遲到判定與扣薪開關可獨立控制（如啟用遲到判定但不扣薪，僅作記錄用途）。
- **後置條件 (Postconditions)**：班別定義完成，可供排班管理模組引用；考勤打卡模組依據班別設定判定遲到/早退。

活動圖：

UC-ATT-SHIFT-01 活動圖

[UC-ATT-SHIFT-02] 排班管理 (Manage Shift Schedules)

- **參與者 (Actor)**：人資管理員 (HR/Admin)、部門主管 (Manager)
- **前置條件 (Preconditions)**：操作者已登入系統並具備排班管理權限；已定義至少一個啟用中的班別；目標員工處於在職狀態。
- **主要流程 (手動排班)**：
  1. 操作者進入「考勤管理 > 排班管理」模組，選擇部門與排班月份。
  2. 系統展示月曆格式的排班表，列出該部門所有員工。
  3. 操作者針對特定員工的特定日期，指派班別（建立 `ShiftSchedule` 聚合根，狀態為 `DRAFT`）。
  4. 操作者可加註備註（`note`）說明特殊排班原因。
  5. 操作者完成排班後點擊「發佈排班」，系統呼叫 `ShiftSchedule.publish()` 將狀態從 `DRAFT` 變更為 `PUBLISHED`。
  6. 系統透過 NTF 模組通知受影響員工查閱排班表。
- **主要流程 (自動排班 - 輪班模式)**：
  1. 操作者進入「排班管理 > 輪班模式」，選擇或建立輪班模式（`RotationPattern`）。
  2. 設定輪班模式的循環天數（`cycleDays`）與每天的班別序列（如：早班→中班→晚班→休息，4 天循環）。
  3. 操作者選擇適用的員工群組，設定排班日期範圍與輪班起算日。
  4. 系統呼叫 `ShiftSchedulingDomainService.generateSchedules()` 自動產生排班（跳過休息日）。
  5. 產生的排班初始為 `DRAFT` 狀態，操作者預覽後可微調再發佈。
- **替代流程 (Alternative Flow)**：
  - 操作者可將已發佈的排班退回草稿狀態（呼叫 `ShiftSchedule.revertToDraft()`），重新調整後再發佈。
  - 月結後，系統自動呼叫 `ShiftSchedule.lock()` 鎖定排班（`LOCKED`），不可再修改。
  - 操作者可刪除草稿狀態的排班（呼叫 `ShiftSchedule.markAsDeleted()`）。
- **例外流程 (Exception Flow)**：
  - 若嘗試變更已鎖定的排班，系統拋出 `IllegalStateException`（「已鎖定的排班不可變更」）。
  - 若嘗試發佈非草稿狀態的排班，系統拋出 `IllegalStateException`（「僅草稿狀態可發佈」）。

  - 若嘗試鎖定非已發佈狀態的排班，系統拋出 `IllegalStateException`（「僅已發佈狀態可鎖定」）。
  - 若輪班模式的天序未設定或數量與循環天數不符，系統拋出 `IllegalArgumentException`。
- **商業規則 (Business Rules)**：
  - BR-ATT-SHIFT-02-01：排班狀態流轉為 `DRAFT → PUBLISHED → LOCKED`，不可跳級。
  - BR-ATT-SHIFT-02-02：同一員工同一日期不可重複排班。
  - BR-ATT-SHIFT-02-03：輪班模式的天序數量必須等於循環天數（`days.size() == cycleDays`）。
  - BR-ATT-SHIFT-02-04：月結鎖定後的排班為最終版，作為考勤比對與薪資計算的依據。
  - BR-ATT-SHIFT-02-05：排班至少需在生效日前 3 天發佈，確保員工有足夠時間查閱。
- **後置條件 (Postconditions)**：排班表發佈完成，員工可於 ESS 查閱自己的班表；考勤模組依排班表判定出勤狀態。

活動圖：

UC-ATT-SHIFT-02 活動圖

[UC-ATT-SHIFT-03] 換班申請 (Submit Shift Swap Request)

- **參與者 (Actor)**：一般員工 (Employee)、部門主管 (Manager)
- **前置條件 (Preconditions)**：申請人與交換對象皆處於在職狀態；雙方在申請的日期皆有已發佈的排班記錄。
- **主要流程 (Main Flow)**：
  1. 員工 A 進入「考勤管理 > 換班申請」模組，選擇欲換班的日期與交換對象員工 B。
  2. 系統顯示雙方在指定日期的班別資訊。
  3. 員工 A 填寫換班原因後提交申請，系統建立 `ShiftSwapRequest` 聚合根，初始狀態為 `PENDING_COUNTERPART`。
  4. 系統透過 NTF 模組通知員工 B 有換班邀請。
  5. 員工 B 查閱換班詳情後，選擇「同意」或「拒絕」。
  6. 若員工 B 同意（呼叫 `ShiftSwapRequest.acceptByCounterpart()`），狀態變更為 `PENDING_APPROVAL`，系統通知主管進行審核。
  7. 主管查閱換班申請後，選擇「核准」或「駁回」。
  8. 若主管核准（呼叫 `ShiftSwapRequest.approve(approverId)`），系統呼叫 `ShiftSchedulingDomainService.executeSwap()` 交換雙方排班的班別，狀態變更為 `APPROVED`。
  9. 系統透過 NTF 模組通知雙方員工換班已生效。
- **替代流程 (Alternative Flow)**：
  - 員工 A 可在對方尚未回應或主管尚未審核前取消申請（呼叫 `ShiftSwapRequest.cancel()`），狀態變更為 `CANCELLED`。
  - 若員工 B 拒絕（呼叫 `ShiftSwapRequest.rejectByCounterpart(reason)`），狀態變更為 `REJECTED`，系統通知員工 A。

  - 若主管駁回（呼叫 `ShiftSwapRequest.reject(approverId, reason)`），狀態變更為 `REJECTED`，系統通知雙方員工。
- **例外流程 (Exception Flow)**：
  - 若申請人與交換對象為同一人，系統拋出 `IllegalArgumentException`（「不可與自己換班」）。
  - 若申請人 ID 或交換對象 ID 為空，系統拋出 `IllegalArgumentException`。
  - 若排班日期為空，系統拋出 `IllegalArgumentException`（「排班日期不可為空」）。
  - 若嘗試在非等待狀態執行操作（如已核准後再次核准），系統拋出 `IllegalStateException`。
  - 若嘗試取消已結案的申請（`APPROVED` 或 `REJECTED`），系統拋出 `IllegalStateException`（「已結案的申請不可取消」）。
- **商業規則 (Business Rules)**：
  - BR-ATT-SHIFT-03-01：換班申請狀態流轉為 `PENDING_COUNTERPART → PENDING_APPROVAL → APPROVED/REJECTED`，另可在任何非結案狀態下 `CANCELLED`。
  - BR-ATT-SHIFT-03-02：換班只能在排班未鎖定的狀態下執行（`ScheduleStatus != LOCKED`）。
  - BR-ATT-SHIFT-03-03：換班不影響雙方的工時統計，僅交換班別。
  - BR-ATT-SHIFT-03-04：換班申請需在排班日期前至少 1 天提出。
  - BR-ATT-SHIFT-03-05：同一員工在同一日期不可同時有多筆進行中的換班申請。
- **後置條件 (Postconditions)**：換班核准後，雙方排班表更新為交換後的班別；考勤打卡依新班別判定出勤狀態。

活動圖：

UC-ATT-SHIFT-03 活動圖

核心業務流程圖 (Business Flowcharts & Swimlane)

本文件展示系統中跨模組、高複雜度的核心業務邏輯流程。透過泳道圖 (Swimlane Diagram) 清晰界定出不同角色 (Actors) 以及後端微服務集群 (Microservices) 於各個階段應負擔的責任與系統邊界條件。

一、 考勤結算至薪資發放流程 (Attendance to Payroll Flow)

此流程涵蓋了企業人資系統中最關鍵的「月結處理」。它不僅牽涉前端使用者的互動，更涉及【考勤模組 (ATT)】與【薪資模組 (PAY)】之間的跨服務資料同步與非同步事件運算。

> *圖表格式： PlantUML 渲染 | 原始碼： `04_核心業務流程圖.puml` | 渲染指令： `java -jar tools/plantuml.jar -smetana -o diagrams 04_核心業務流程圖.puml`*

考勤結算至薪資發放流程

流程設計要點 (Architecture Design Points)

此設計展示了複雜的分散式結算邏輯如何被穩健地處理：

1. **防呆與依賴檢查 (Constraint & Validation)**：在進入薪資計算（Phase 3）前，必須確保考勤資料已經完全鎖定（Phase 2）。系統防呆設計會阻擋在仍有請假單卡在主管端的狀態下進行結算，確保資料一致性 (Data Integrity)。
2. **事件與 API Query 的混合應用 (Hybrid Communication)**：
   - 考勤結班後會發出 Event 告知 PAY 模組「這段期間的資料已鎖定」。
   - 但計薪時，基於資料即時性且為了避免把龐大運算參數塞入 Kafka 訊息中，PAY 模組是主動呼叫 ATT 與 INS 的 Query API 索取精確的時數與級距數字（API Composition 模式）。此做法降低了 Queue 的負載並提高了資料獲取的正確性。

二、員工入職流程 (Employee Onboarding Flow)

此流程描述從 HR 建立員工資料到完成所有系統初始化的跨服務協作。涉及【組織模組 (ORG)】、【IAM 模組】、【保險模組 (INS)】、【薪資模組 (PAY)】四個服務的非同步事件串接。

```
PlantUML 渲染失敗
```

流程設計要點

1. **事件驅動解耦（Event-Driven Decoupling）**：HR02 只負責建立員工本身，其餘服務透過訂閱 `EmployeeCreatedEvent` 自動完成後續作業，實現服務間零耦合。
2. **最終一致性（Eventual Consistency）**：三個下游服務（IAM、Insurance、Payroll）平行處理，任一服務失敗不影響其他服務，透過 Dead Letter Queue 機制確保最終一致。
3. **冪等性設計（Idempotency）**：每個訂閱者以 `employee_id` 作為冪等鍵，重複消費同一事件不會產生重複資料。

三、請假簽核流程 (Leave Approval Flow)

此流程展示員工發起請假到主管核准的完整路徑，涉及【考勤模組 (ATT)】與【簽核模組 (WFL)】的協作。

```
PlantUML 渲染失敗
```

流程設計要點

1. **預扣機制（Reservation Pattern）**：申請時先預扣餘額避免超額申請，駁回時退還。此模式類似電商的庫存預留。
2. **動態簽核路由（Dynamic Routing）**：Workflow 服務根據組織結構樹自動決定簽核鏈，支援多層簽核與代理人機制。
3. **狀態機驅動（State Machine）**：LeaveRequest 的生命週期由狀態機管理：`PENDING → APPROVED/REJECTED`，確保狀態轉換的合法性。

四、績效考核流程 (Performance Review Flow)

此流程展示一個完整的績效週期，從 HR 建立考核週期到最終確認評等，涉及【績效模組 (PER)】的多角色協作。

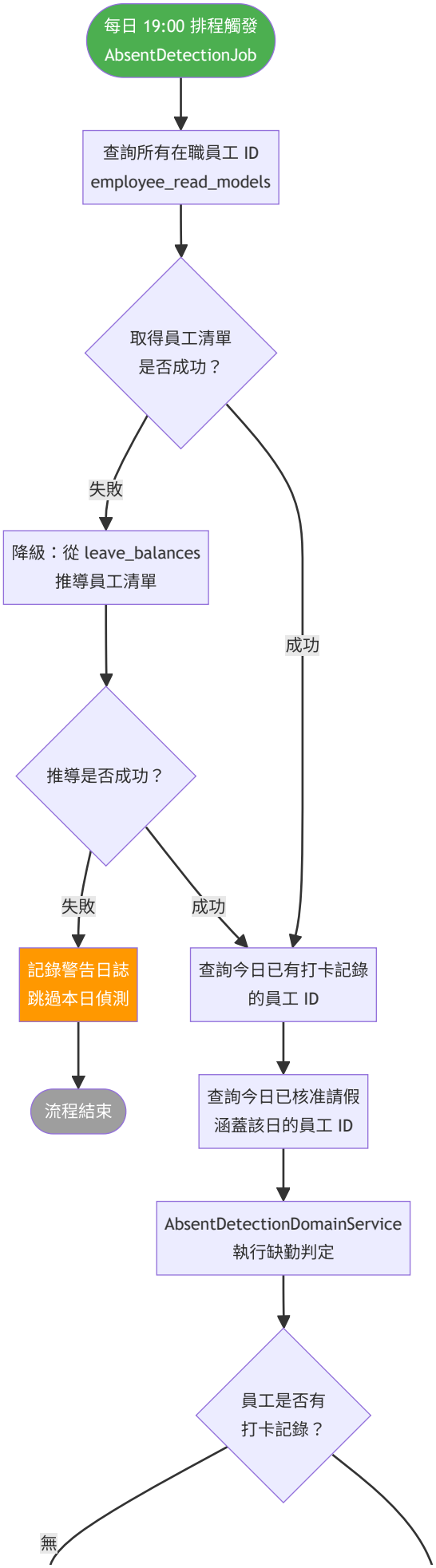
```
PlantUML 渲染失敗
```

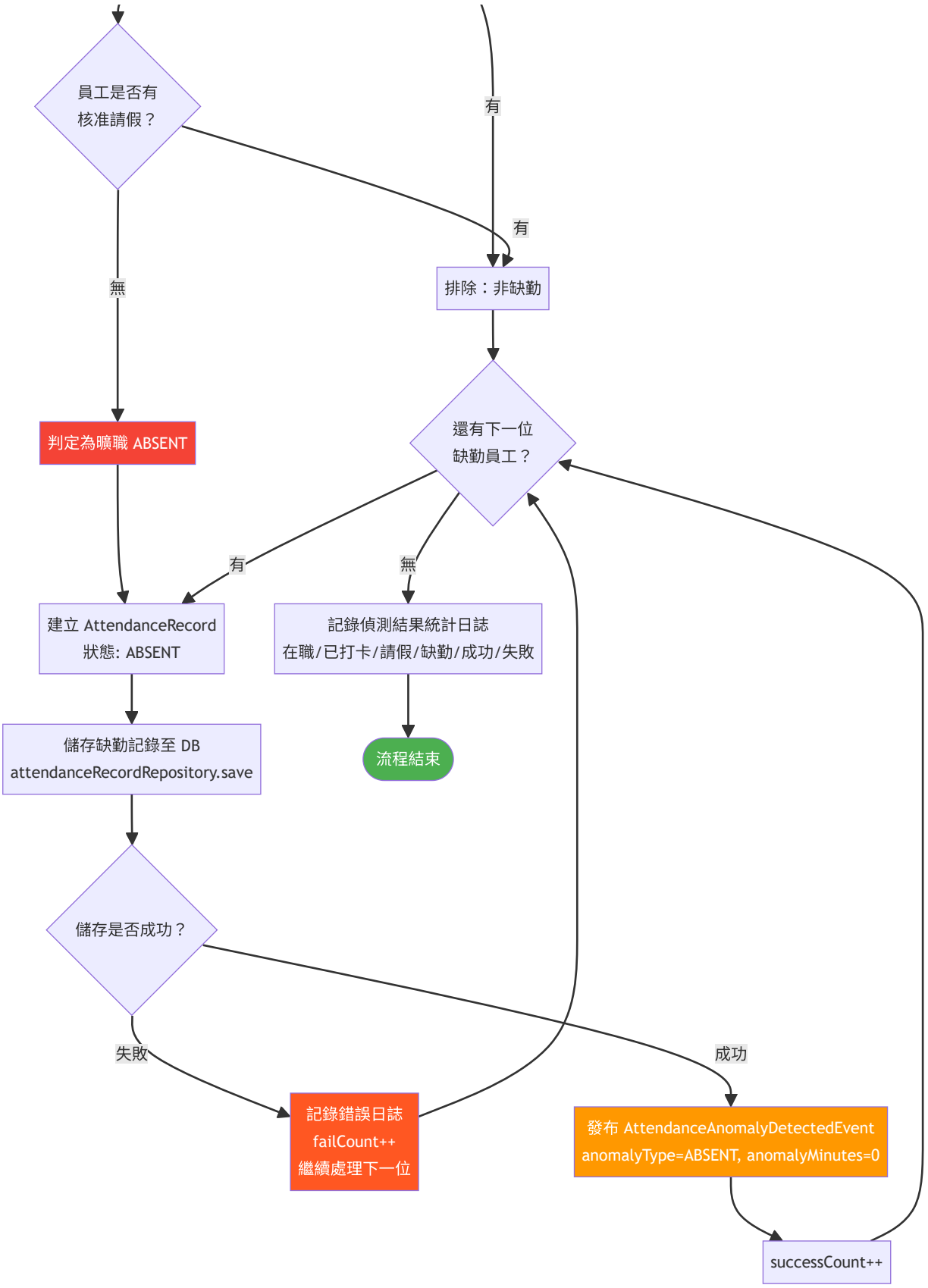
流程設計要點

1. **彈性表單設計（Flexible Form）**：考核項目與權重由 HR 於週期建立時自定義，支援 KPI、OKR、360 度等多種考核類型。
2. **狀態機嚴格控制（State Machine）**：表單狀態依序流轉 `ACTIVE → SELF_REVIEWED → MANAGER_REVIEWED → FINALIZED`，不可跳步或回退，確保流程正確性。
3. **強制分配機制（Forced Distribution）**：HR 可選擇啟用各評等的人數比例限制（如 A 等不超過 20%），系統會在確認階段提供分佈警示。

五、曠職判定流程 (Absent Detection Flow)

此流程描述系統如何透過每日排程自動偵測未出勤員工並判定曠職，涉及【考勤模組 (ATT)】的排程任務與領域服務協作，以及透過 Domain Event 通知相關服務。


每日 19:00 排程觸發
AbsentDetectionJob
查詢所有在職員工 ID
employee_read_models
取得員工清單
是否成功？
失敗
成功
降級：從 leave_balances
推導員工清單
推導是否成功？
失敗
成功
記錄警告日誌
跳過本日偵測
查詢今日已有打卡記錄
的員工 ID
流程結束
查詢今日已核准請假
涵蓋該日的員工 ID
AbsentDetectionDomainService
執行缺勤判定
員工是否有
打卡記錄？
無



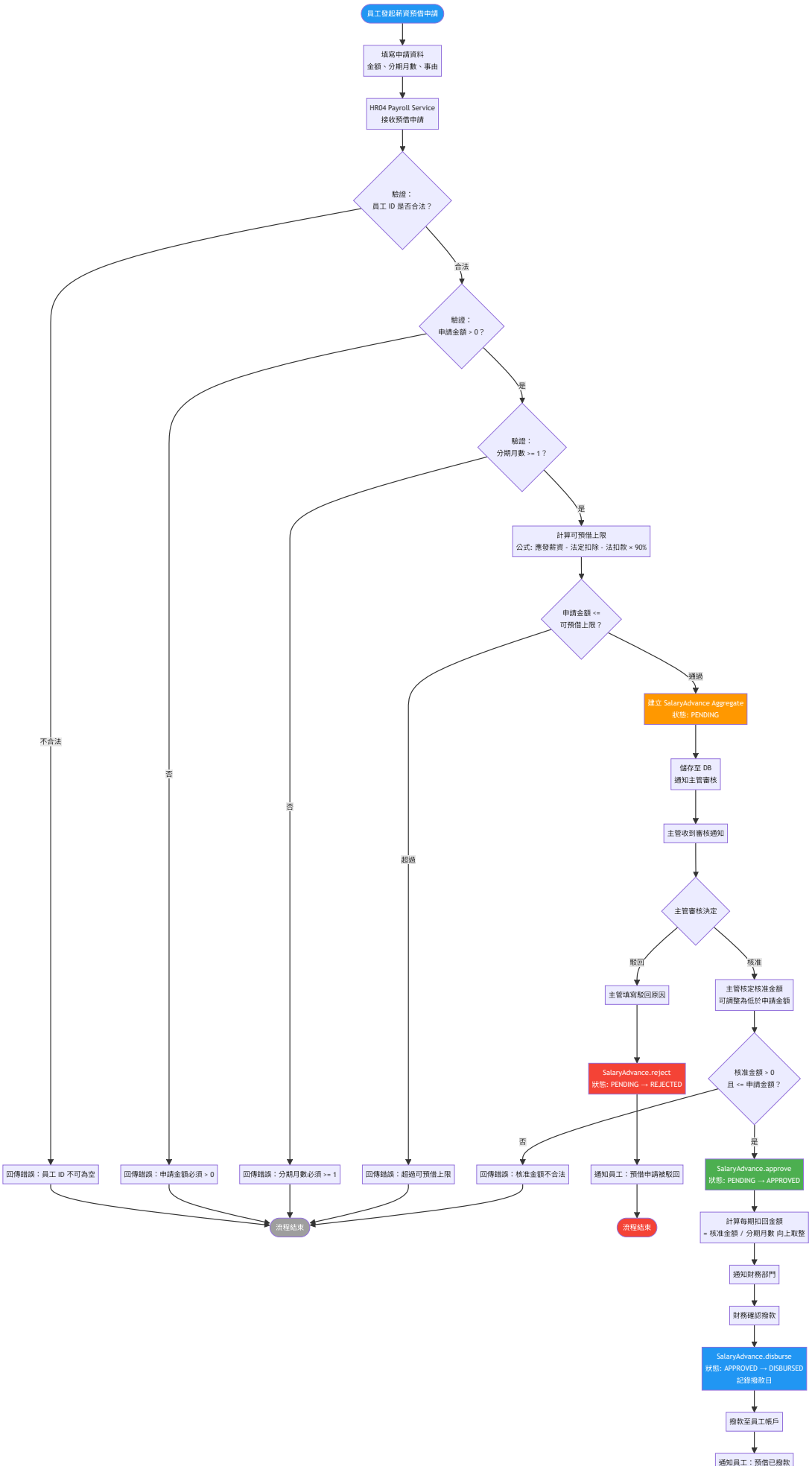
流程設計要點

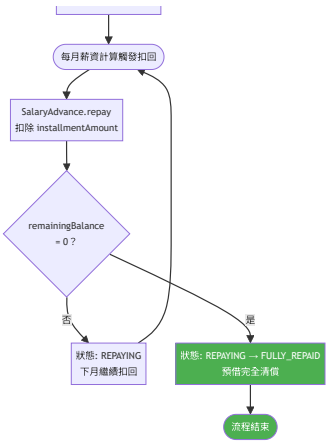
1. **排程驅動（Scheduled Job）**：透過 `AbsentDetectionJob` 於每日 19:00 自動執行，以 Spring `@Scheduled(cron = "0 0 19 * * ?")` 宣告，確保每日工作結束後統一偵測。

2. **三層過濾邏輯（Domain Service）**：`AbsentDetectionDomainService` 負責核心判定 —— 從全部在職員工中排除「已打卡」與「已核准請假」者，剩餘即為曠職。此邏輯為純 POJO，不依賴框架。
3. **降級容錯（Fallback Strategy）**：若 `employee_read_models` 表不存在（非 CQRS 環境），自動降級從 `leave_balances` 推導在職員工清單，確保排程不因跨服務表缺失而中斷。
4. **逐筆異常隔離（Per-Record Error Isolation）**：單一員工的缺勤記錄建立失敗不影響其他員工的處理，透過 try-catch 逐筆隔離，最後統計成功/失敗數量。
5. **事件通知（Event-Driven）**：每筆曠職記錄建立後發布 `AttendanceAnomalyDetectedEvent`，下游服務（如通知模組）可訂閱此事件推送給主管。

六、薪資預借審核流程 (Salary Advance Approval Flow)

此流程描述員工申請薪資預借，經過資格與額度檢查、主管審核、財務撥款，到每月自動扣回的完整生命週期。涉及【薪資模組 (PAY)】的 SalaryAdvance 聚合根狀態機。


員工發起薪資預借申請
填寫申請資料
金額、分期月數、事由
HR04 Payroll Service
接收預借申請
驗證：
員工 ID 是否合法？
合法
不合法
回傳錯誤：員工 ID 不可為空
驗證：
申請金額 > 0？
是
否
回傳錯誤：申請金額必須 > 0
驗證：
分期月數 >= 1？
是
否
回傳錯誤：分期月數必須 >= 1
計算可預借上限
公式: 應發薪資 - 法定扣除 - 法扣款 × 90%
申請金額 <=
可預借上限？
通過
超過
回傳錯誤：超過可預借上限
建立 SalaryAdvance Aggregate
狀態: PENDING
儲存至 DB
通知主管審核
主管收到審核通知
主管審核決定
駁回
核准
主管填寫駁回原因
SalaryAdvance.reject
狀態: PENDING → REJECTED
通知員工：預借申請被駁回
流程結束
主管核定核准金額
可調整為低於申請金額
核准金額 > 0
且 <= 申請金額？
否
是
回傳錯誤：核准金額不合法
SalaryAdvance.approve
狀態: PENDING → APPROVED
計算每期扣回金額
= 核准金額 / 分期月數 向上取整
通知財務部門
財務確認撥款
SalaryAdvance.disburse
狀態: APPROVED → DISBURSED
記錄撥款日
撥款至員工帳戶
通知員工：預借已撥款
流程結束

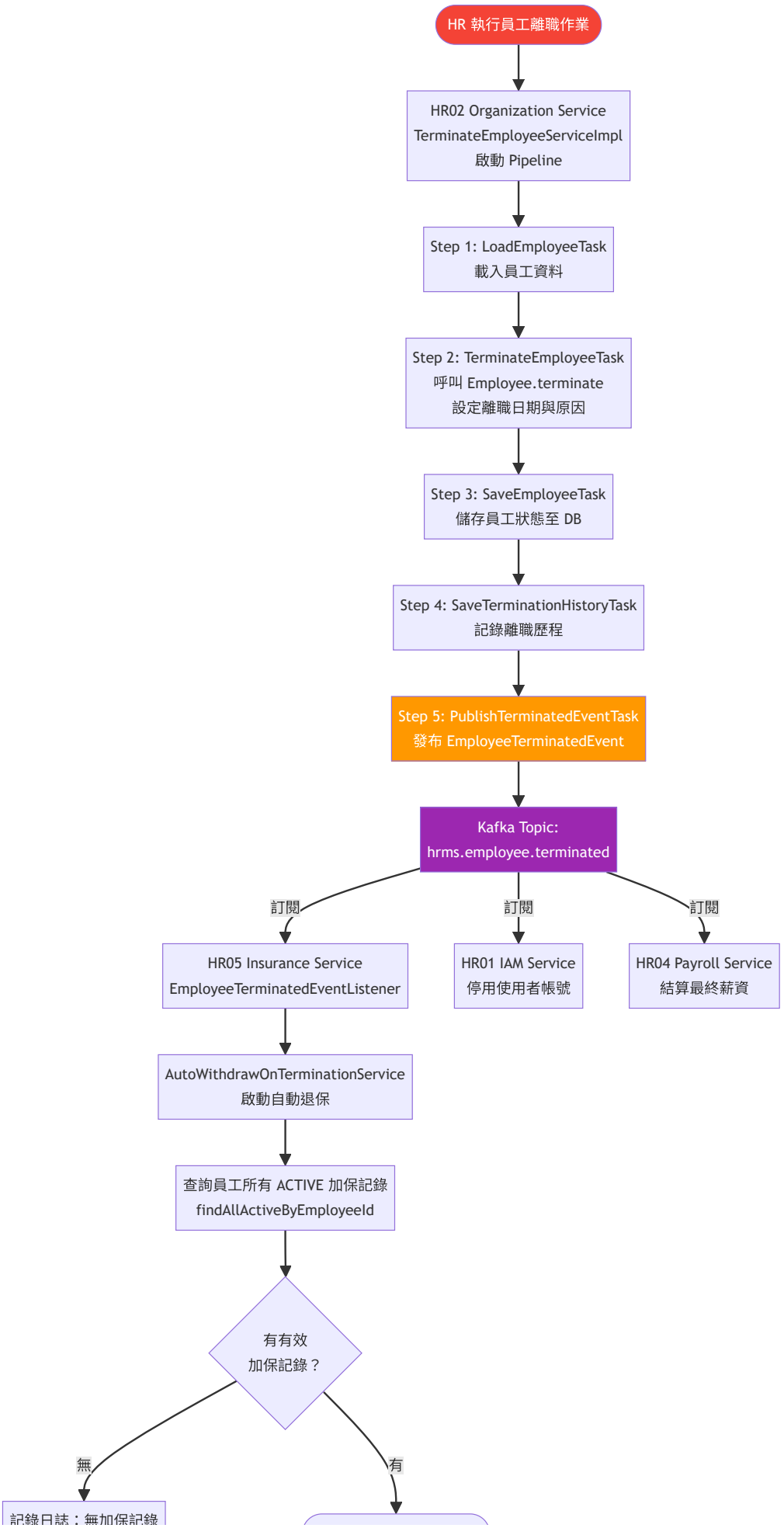


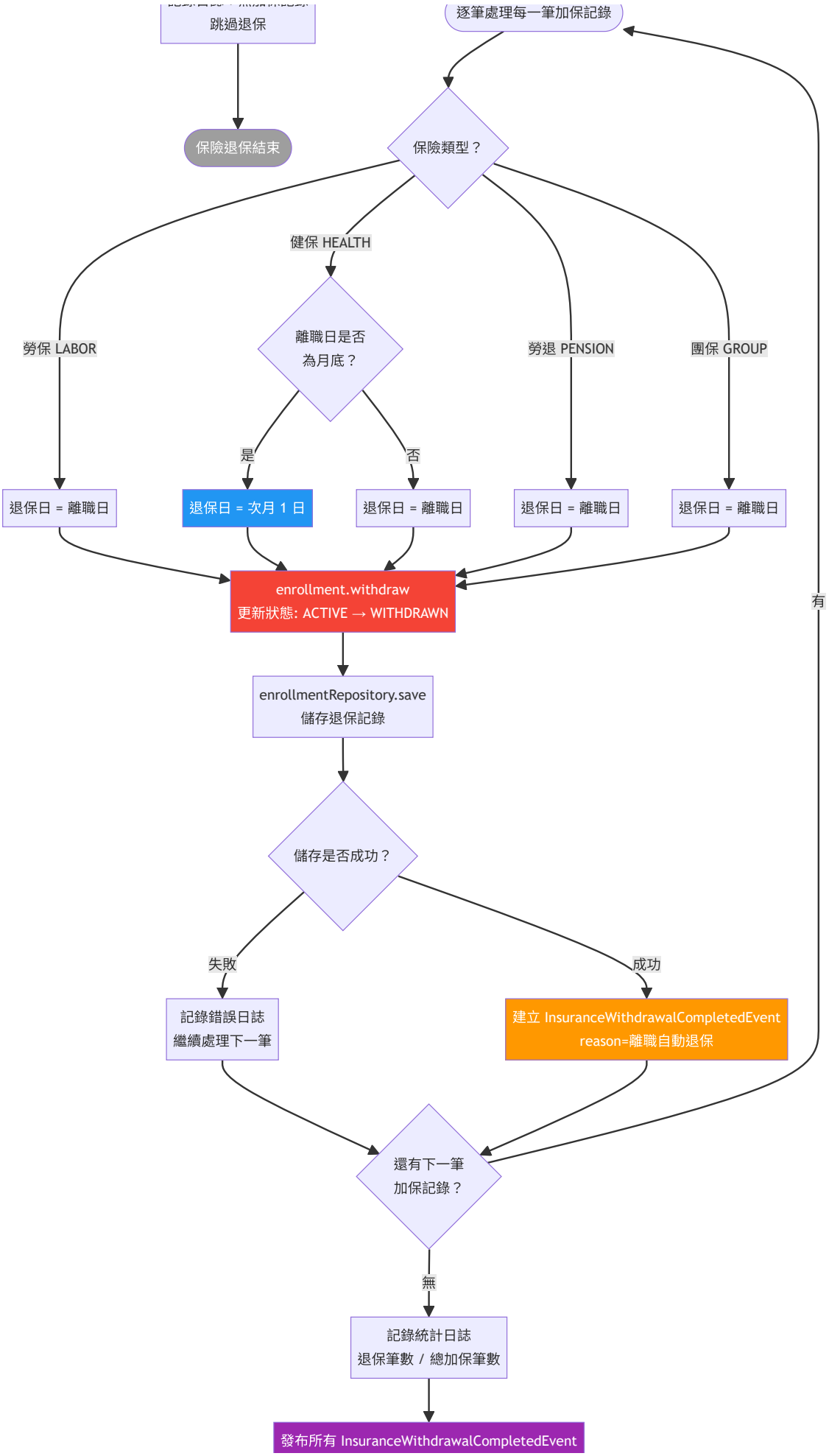
流程設計要點

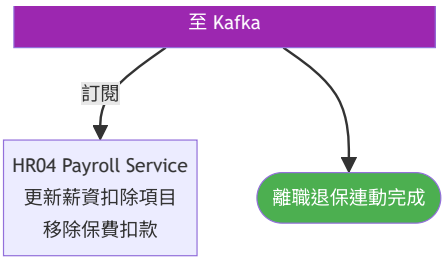
1. **聚合根狀態機（Aggregate State Machine）**：`SalaryAdvance` 聚合根嚴格管控狀態轉換 `PENDING → APPROVED → DISBURSED → REPAYING → FULLY_REPAID`，每個方法在入口處驗證當前狀態，不合法的轉換拋出 `IllegalStateException`。另有 `REJECTED` 與 `CANCELLED` 兩個終態。
2. **預借額度公式（Max Advance Calculation）**：`可預借金額 <= (應發薪資 - 法定扣除 - 法扣款) × 90%`，確保員工扣回預借後仍有基本薪資保障。此公式以靜態方法 `SalaryAdvance.calculateMaxAdvance()` 實現。
3. **分期扣回機制（Installment Repayment）**：核准後自動計算 `installmentAmount = approvedAmount / installmentMonths`（向上取整），每月薪資計算時自動呼叫 `repay()` 扣回，最後一期扣回金額 = `remainingBalance`，確保不多扣。
4. **核准金額可調整（Adjustable Approval）**：主管可核准低於申請金額的額度（但不可超過申請金額），提供彈性審核空間。
5. **取消限制（Cancel Constraint）**：僅 `PENDING` 與 `APPROVED` 狀態可取消，已撥款（`DISBURSED`）或扣回中（`REPAYING`）的預借不可取消，保護財務一致性。

七、離職退保連動流程 (Termination Insurance Withdrawal Flow)

此流程描述員工離職生效後，系統自動觸發退保的跨服務事件驅動流程。涉及【組織模組 (ORG)】、【保險模組 (INS)】、【IAM 模組】、【薪資模組 (PAY)】四個服務的非同步協作。


HR 執行員工離職作業
HR02 Organization Service
TerminateEmployeeServiceImpl
啟動 Pipeline
Step 1: LoadEmployeeTask
載入員工資料
Step 2: TerminateEmployeeTask
呼叫 Employee.terminate
設定離職日期與原因
Step 3: SaveEmployeeTask
儲存員工狀態至 DB
Step 4: SaveTerminationHistoryTask
記錄離職歷程
Step 5: PublishTerminatedEventTask
發布 EmployeeTerminatedEvent
Kafka Topic:
hrms.employee.terminated
訂閱
訂閱
訂閱
HR05 Insurance Service
EmployeeTerminatedEventListener
HR01 IAM Service
停用使用者帳號
HR04 Payroll Service
結算最終薪資
AutoWithdrawOnTerminationService
啟動自動退保
查詢員工所有 ACTIVE 加保記錄
findAllActiveByEmployeeId
有有效
加保記錄？
無
有

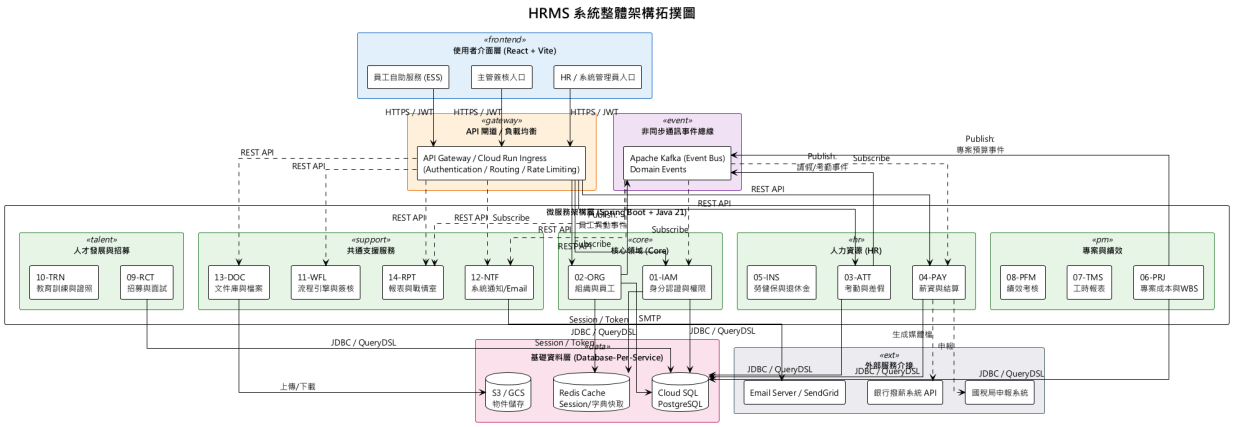

跳過退保
逐筆處理每一筆加保記錄
保險退保結束
保險類型？
健保 HEALTH
勞保 LABOR
勞退 PENSION
團保 GROUP
離職日是否
為月底？
是
否
退保日 = 離職日
退保日 = 次月 1 日
退保日 = 離職日
退保日 = 離職日
退保日 = 離職日
enrollment.withdraw
更新狀態: ACTIVE → WITHDRAWN
enrollmentRepository.save
儲存退保記錄
儲存是否成功？
失敗
成功
記錄錯誤日誌
繼續處理下一筆
建立 InsuranceWithdrawalCompletedEvent
reason=離職自動退保
還有下一筆
加保記錄？
有
無
記錄統計日誌
退保筆數 / 總加保筆數
發布所有 InsuranceWithdrawalCompletedEvent



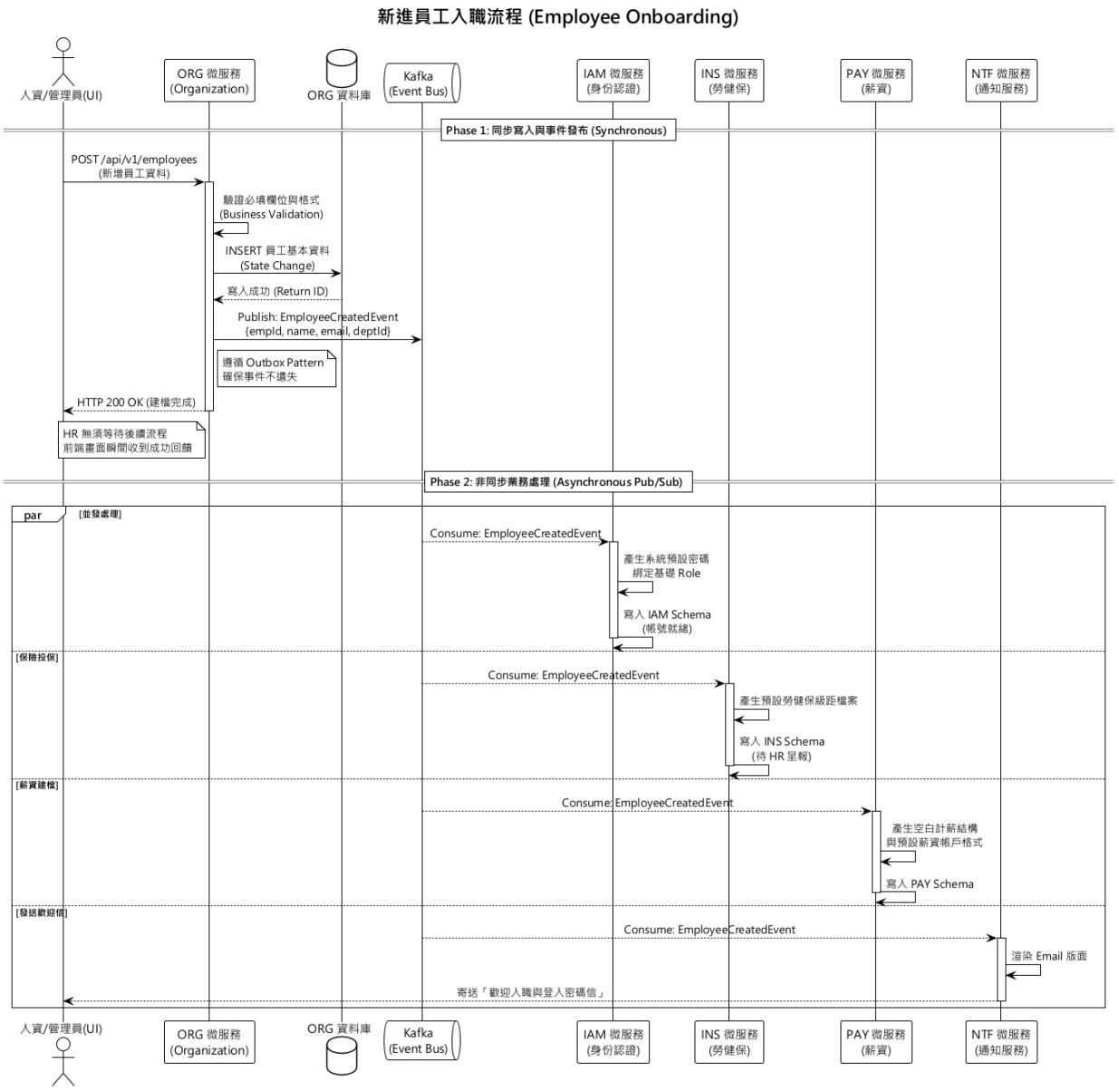
流程設計要點

1. **Pipeline 編排（Business Pipeline）**：離職作業由 `TerminateEmployeeServiceImpl` 透過 `BusinessPipeline` 依序執行 5 個 Task（載入 → 離職 → 儲存 → 歷程 → 事件），符合 Application Service 只做編排不做決策的原則。
2. **事件驅動跨服務連動（Event-Driven Cross-Service）**：`EmployeeTerminatedEvent` 攜帶 `employeeId`、`terminationDate`、`terminationReason` 等完整資訊，透過 Kafka topic `hrms.employee.terminated` 廣播，保險/IAM/薪資服務各自訂閱處理。
3. **退保日期規則引擎（EffectiveDateRule）**：依保險類型自動計算退保生效日 —— 勞保/勞退/團保為離職日退保；健保特殊規則：月底離職者次月 1 日退保。此規則封裝於 `EffectiveDateRule.calculateWithdrawDate()` 值物件中。
4. **逐筆隔離與降級（Per-Record Isolation）**：每筆加保記錄的退保獨立處理，單筆退保失敗不影響其他記錄，透過 try-catch 隔離錯誤並記錄日誌。
5. **雙層事件鏈（Two-Layer Event Chain）**：第一層 `EmployeeTerminatedEvent`（ORG → INS）觸發退保；第二層 `InsuranceWithdrawalCompletedEvent`（INS → PAY）通知薪資模組移除保費扣款項目，實現完整的事件驅動解耦。
6. **冪等性保障（Idempotency）**：`AutoWithdrawOnTerminationService` 查詢的是 `ACTIVE` 狀態的加保記錄，已退保（`WITHDRAWN`）的記錄不會被重複處理，確保事件重複消費的安全性。

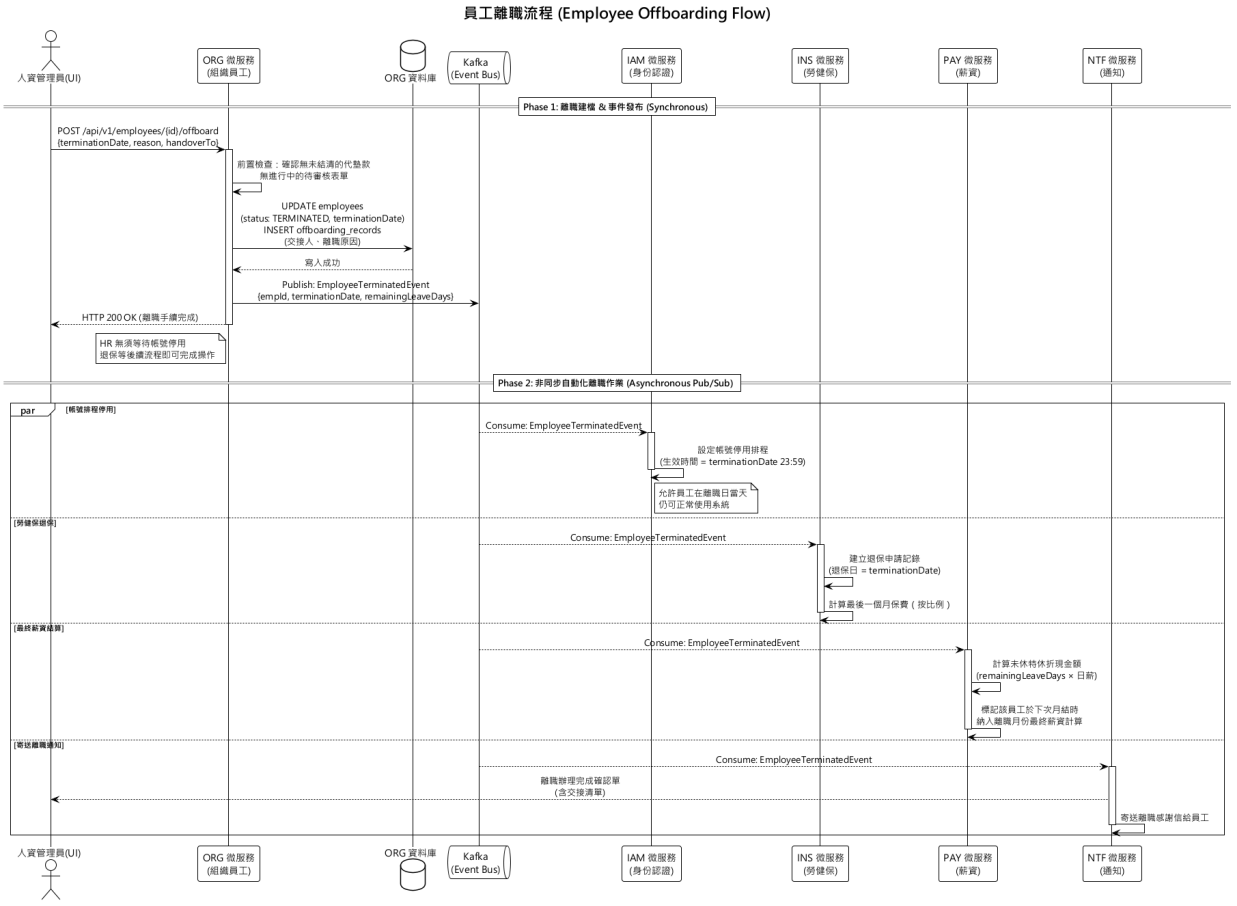
架構圖表



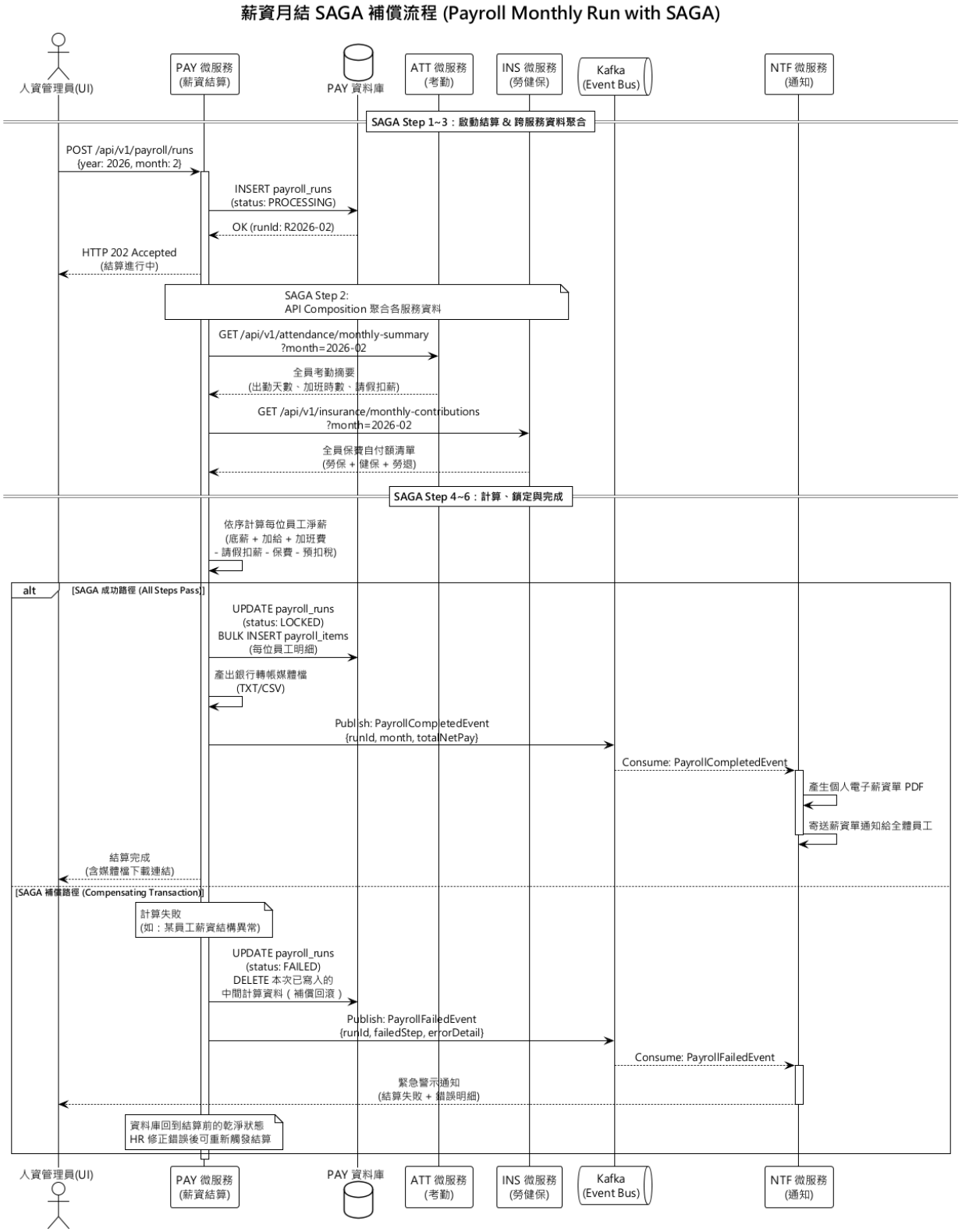
00 系統架構拓撲圖


新進員工入職流程 (Employee Onboarding)
人資/管理員(UI)
ORG 微服務 (Organization)
ORG 資料庫
Kafka (Event Bus)
IAM 微服務 (身份認證)
INS 微服務 (勞健保)
PAY 微服務 (薪資)
NTF 微服務 (通知服務)
Phase 1: 同步寫入與事件發布 (Synchronous)
POST /api/v1/employees (新增員工資料)
驗證必填欄位與格式 (Business Validation)
INSERT 員工基本資料 (State Change)
寫入成功 (Return ID)
Publish: EmployeeCreatedEvent {empId, name, email, deptId}
遵循 Outbox Pattern 確保事件不遺失
HTTP 200 OK (建檔完成)
HR 無須等待後續流程 前端畫面瞬間收到成功回饋
Phase 2: 非同步業務處理 (Asynchronous Pub/Sub)
par
[並發處理]
Consume: EmployeeCreatedEvent
產生系統預設密碼 綁定基礎 Role
寫入 IAM Schema (帳號就緒)
[保險投保]
Consume: EmployeeCreatedEvent
產生預設勞健保級距檔案
寫入 INS Schema (待 HR 呈報)
[薪資建檔]
Consume: EmployeeCreatedEvent
產生空白計薪結構 與預設薪資帳戶格式
寫入 PAY Schema
[發送歡迎信]
Consume: EmployeeCreatedEvent
渲染 Email 版面
寄送「歡迎入職與登入密碼信」

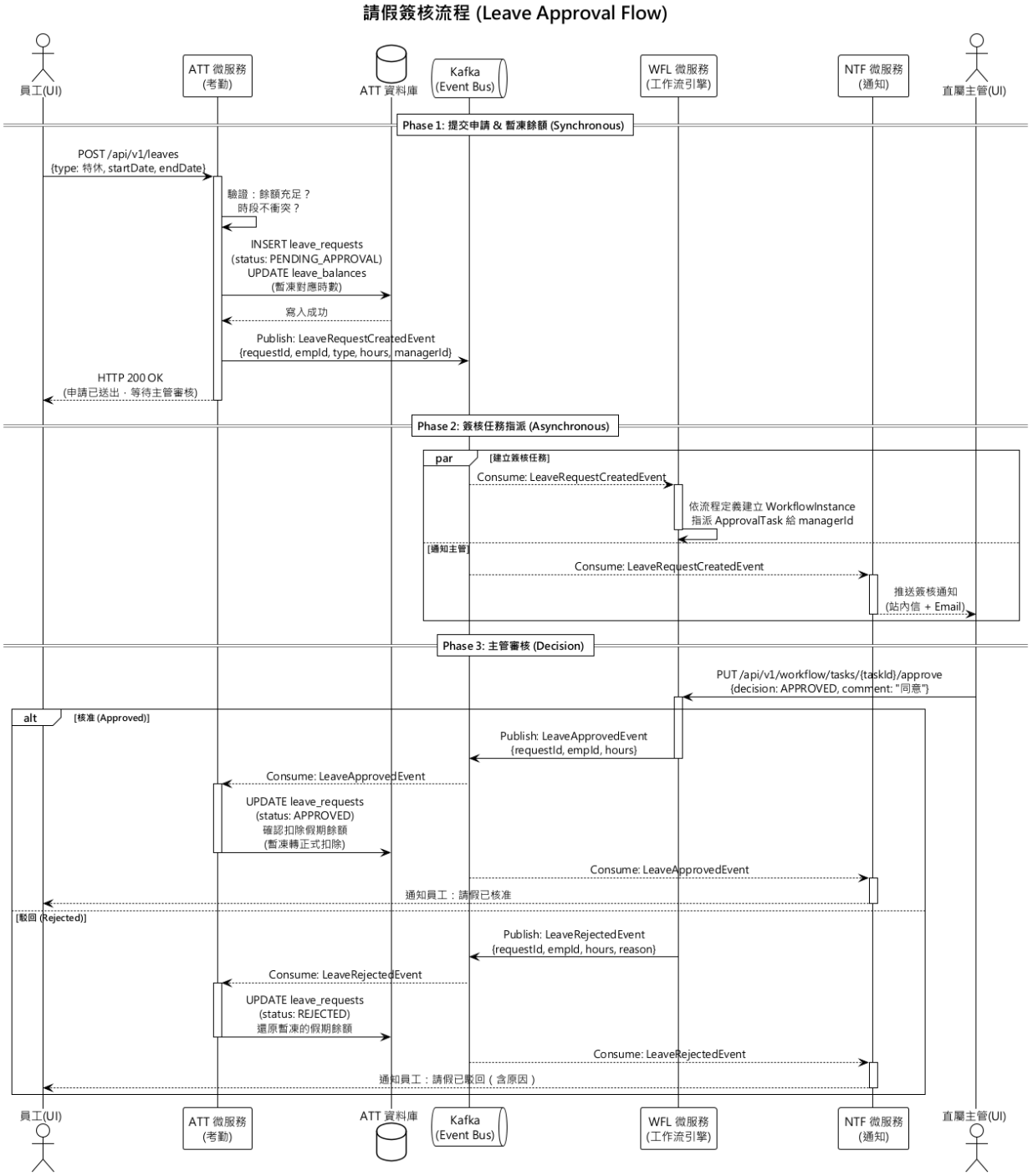
01 員工入職流程


員工離職流程 (Employee Offboarding Flow)
人資管理員(UI)
ORG 微服務 (組織員工)
ORG 資料庫
Kafka (Event Bus)
IAM 微服務 (身份認證)
INS 微服務 (勞健保)
PAY 微服務 (薪資)
NTF 微服務 (通知)
Phase 1: 離職建檔 & 事件發布 (Synchronous)
POST /api/v1/employees/{id}/offboard (terminationDate, reason, handoverTo)
前置檢查：確認無未結清的代墊款 無進行中的待審核表單
UPDATE employees (status: TERMINATED, terminationDate) INSERT offboarding_records (交接人、離職原因)
寫入成功
Publish: EmployeeTerminatedEvent {empId, terminationDate, remainingLeaveDays}
HTTP 200 OK (離職手續完成)
HR 無須等待帳號停用 退保等後續流程即可完成操作
Phase 2: 非同步自動化離職作業 (Asynchronous Pub/Sub)
par
[帳號排程停用]
Consume: EmployeeTerminatedEvent
設定帳號停用排程 (生效時間 = terminationDate 23:59)
允許員工在離職日當天 仍可正常使用系統
[勞健保退保]
Consume: EmployeeTerminatedEvent
建立退保申請記錄 (退保日 = terminationDate)
計算最後一個月保費（按比例）
[最終薪資結算]
Consume: EmployeeTerminatedEvent
計算未休特休折現金額 (remainingLeaveDays × 日薪)
標記該員工於下次月結時 納入離職月份最終薪資計算
[寄送離職通知]
Consume: EmployeeTerminatedEvent
離職辦理完成確認單 (含交接清單)
寄送離職感謝信給員工

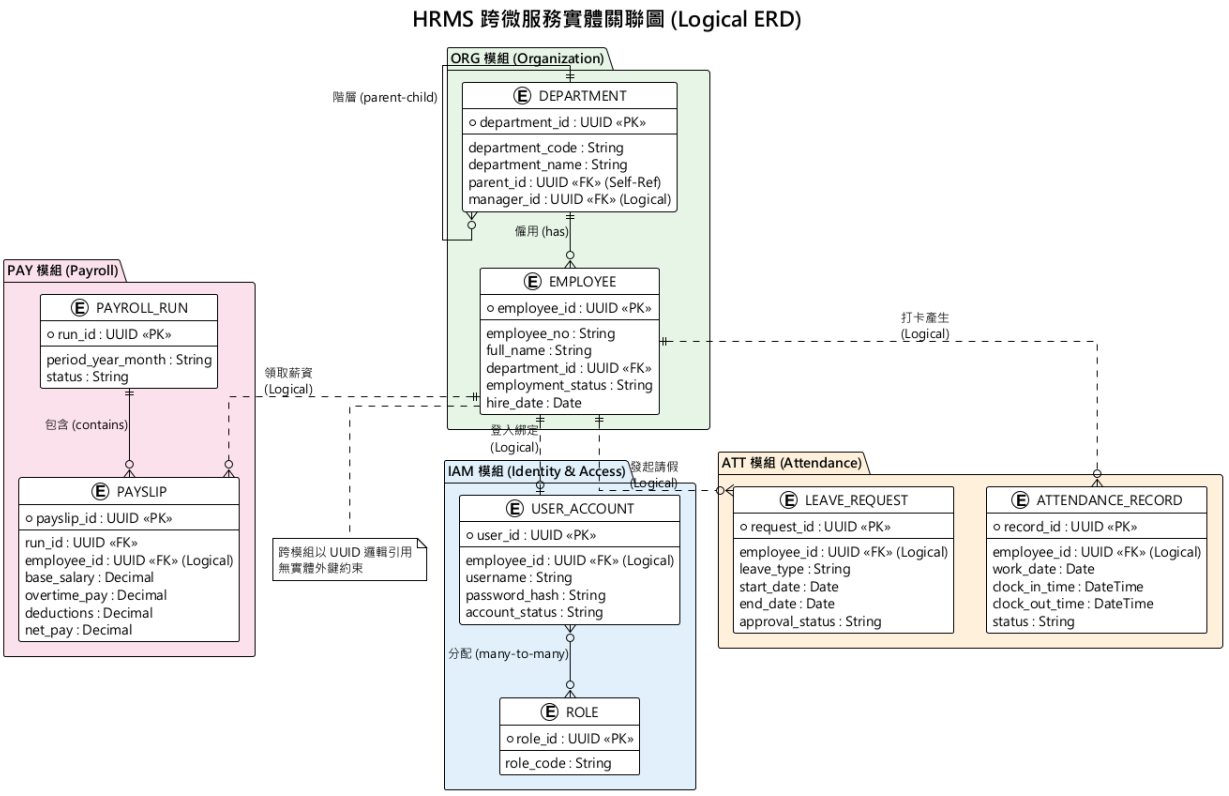
01 員工離職流程



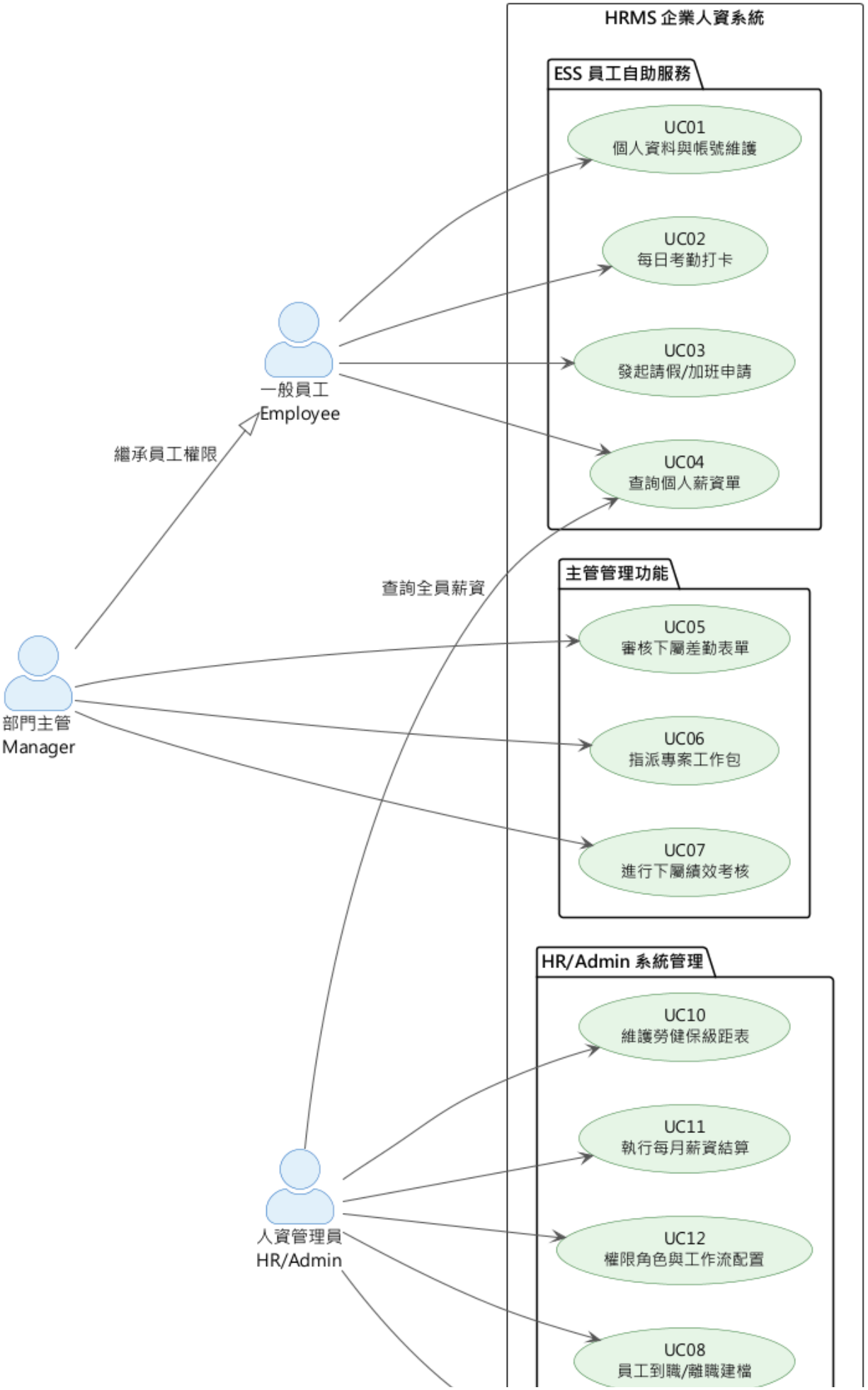
01 薪資SAGA補償流程

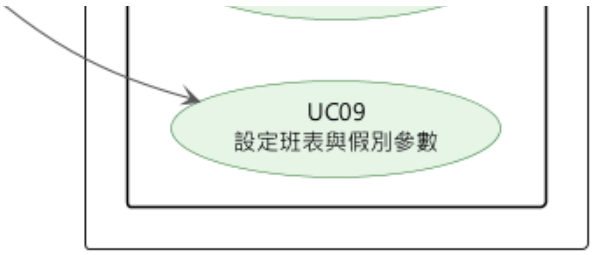

請假簽核流程 (Leave Approval Flow)
員工(UI)
ATT 微服務 (考勤)
ATT 資料庫
Kafka (Event Bus)
WFL 微服務 (工作流引擎)
NTF 微服務 (通知)
直屬主管(UI)
Phase 1: 提交申請 & 暫凍餘額 (Synchronous)
POST /api/v1/leaves {type: 特休, startDate, endDate}
驗證：餘額充足？ 時段不衝突？
INSERT leave_requests (status: PENDING_APPROVAL) UPDATE leave_balances (暫凍對應時數)
寫入成功
Publish: LeaveRequestCreatedEvent {requestId, empId, type, hours, managerId}
HTTP 200 OK (申請已送出，等待主管審核)
Phase 2: 簽核任務指派 (Asynchronous)
par
[建立簽核任務]
Consume: LeaveRequestCreatedEvent
依流程定義建立 WorkflowInstance 指派 ApprovalTask 給 managerId
[通知主管]
Consume: LeaveRequestCreatedEvent
推送簽核通知 (站內信 + Email)
Phase 3: 主管審核 (Decision)
PUT /api/v1/workflow/tasks/{taskId}/approve {decision: APPROVED, comment: "同意"}
alt
[核准 (Approved)]
Publish: LeaveApprovedEvent {requestId, empId, hours}
Consume: LeaveApprovedEvent
UPDATE leave_requests (status: APPROVED) 確認扣除假期餘額 (暫凍轉正式扣除)
Consume: LeaveApprovedEvent
通知員工：請假已核准
[駁回 (Rejected)]
Publish: LeaveRejectedEvent {requestId, empId, hours, reason}
Consume: LeaveRejectedEvent
UPDATE leave_requests (status: REJECTED) 還原暫凍的假期餘額
Consume: LeaveRejectedEvent
通知員工：請假已駁回（含原因）

01 請假簽核流程



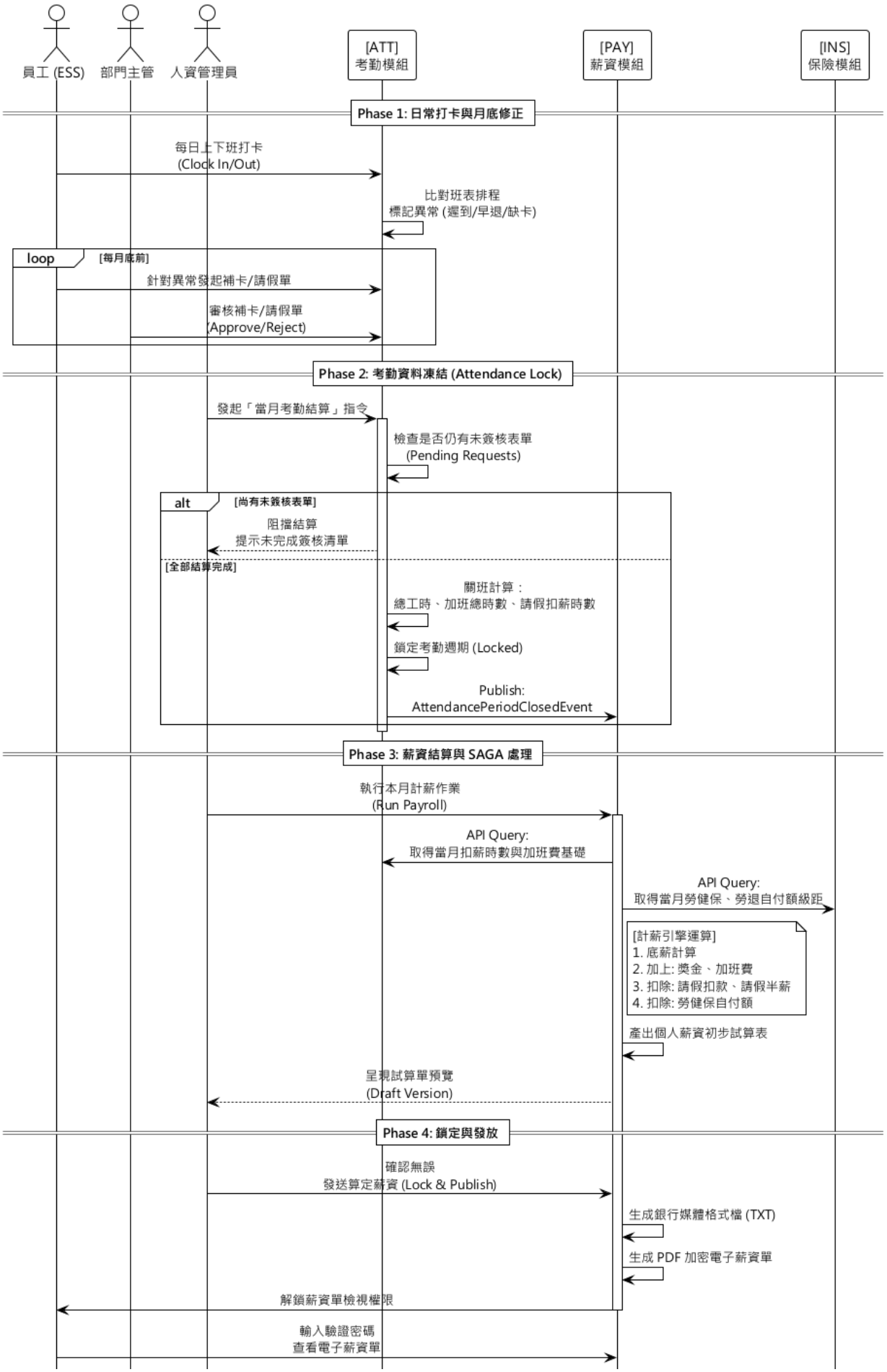
02 跨微服務實體關聯圖


HRMS 企業人資系統
ESS 員工自助服務
UC01
個人資料與帳號維護
UC02
每日考勤打卡
UC03
發起請假/加班申請
UC04
查詢個人薪資單
主管管理功能
UC05
審核下屬差勤表單
UC06
指派專案工作包
UC07
進行下屬績效考核
HR/Admin 系統管理
UC10
維護勞健保級距表
UC11
執行每月薪資結算
UC12
權限角色與工作流配置
UC08
員工到職/離職建檔
一般員工
Employee
繼承員工權限
部門主管
Manager
查詢全員薪資
人資管理員
HR/Admin



03 usecase

考勤結算至薪資發放流程 (Attendance to Payroll Flow)



04 考勤結算至薪資發放流程